

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-a7adc99e0e68455b1.jsonl`  
- SHA-256: `bdfb2d41102011c82ab8d3fd37d9e3278e8e6426d918c73f67fb692111cdd90b`  
- Source modified: `2026-06-18T07:58:34+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_e5140909-890`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T07:54:35.121Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base  
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.  
You are a senior engineer scoping a SAFE refactoring effort. The owner wants critical tech debt  
cleaned up in files that "grew a lot", with ZERO behavior change and NO quality regression.  
Propose only refactors that are: (a) behavior-PRESERVING (pure moves/extractions/dedup/dead-code  
removal ? NOT redesigns, NOT logic changes), (b) backed by EXISTING tests that would catch a  
regression (assess this honestly per proposal ? if coverage is weak/none, say so and rate risk  
higher), (c) cleanly separable into ONE reviewable PR. For each: the exact seam (what moves  
where), the behavior-preservation argument, honest test_coverage, risk, est diff size, value.  
PREFER high-value + low-risk + well-tested. It is FINE to say a giant function should NOT be  
refactored yet because coverage is too thin ? flag that as a 'needs tests first' note in debt.
```

```
Signals (LOC / longest-functions): gemini.process_video=434L/71br, yolo_hybrid=309L,  
trajectory_hybrid=231L,  
motion=204L, compiler.compile_highlights=202L, golden_fixtures=1113L (freeze_real_fixture=186L),  
main.py=871L (main=168L,_execute_processing=112L), database.py=738L (_init_db=187L),  
wasb_infer=699L (run=185L),  
ablation=620L, experiment=579L, tracknet_runner=506L.
```

```
CLUSTER: eval god-modules ?  
backend/eval/{golden_fixtures.py(1113),ablation.py(620),experiment.py(579),rally_seg_eval.py}.  
Look for clean module splits (e.g. PDF/reporting vs core), duplicated logic across eval loaders,  
dead code, and long functions to extract. Check tests/ coverage.
```

2. assistant (2026-06-18T07:54:38.084Z)

I'll start by reading the cluster files to understand their structure, then assess test coverage honestly before proposing refactors.

3. user (2026-06-18T07:54:40.731Z)

```
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
```

```
---  
1113 backend/eval/golden_fixtures.py  
620  backend/eval/ablation.py  
579  backend/eval/experiment.py  
380  backend/eval/rally_seg_eval.py  
2692 total
```

4. user (2026-06-18T07:54:41.817Z)

```
__init__.py  
ablation.py
```

```
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
golden_fixtures.py
gt_loader.py
harness.py
metrics.py
per_user_eval.py
per_user_gate.py
per_user_regression.py
promotion.py
rally_seg_eval.py
rally_seq_proto.py
regression.py
score_calibration.py
segmentation_metrics.py
serve_contrast.py
served_gate_a.py
splits.py
tolerance_metrics.py
training.py
windowing.py
=== TESTS ===
test_ablation.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_experiment.py
test_experiment_honest.py
test_fusion_golden_telemetry.py
test_golden_fixtures.py
test_golden_real_fixtures.py
test_per_user_eval.py
test_rally_seg_eval.py
```

5. assistant (2026-06-18T07:54:43.846Z)

Let me read the four cluster files and their corresponding tests.

6. user (2026-06-18T07:54:45.199Z)

```
1      """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3      `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the single shared
4      replay code path** used by both the freeze tool and the characterization gate, and it
5      deliberately stays on the CONTROL path** (D5 in that doc): the learned
```

```

6      variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7      across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8      only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9      merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10     LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11     `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12     path simply never calls into it.)
13
14     The replay reuses the already-shipped eval stack VERBATIM::
15
16         parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17         ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18         ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
19
20     Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22     * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"``.
25     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27     * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28     passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29     named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30     silently churn fixtures.
31     * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32     the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33     takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34     has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35     ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36     tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38     §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
39     suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
42     field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
43
44     Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
45     parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay

```

```

48     never calls into it.
49     """
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import os
56     import re
57     import shutil
58     import sys
59     import tempfile
60     from dataclasses import dataclass
61     from pathlib import Path
62     from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
65     -----
66     from backend.eval import distill_local
67     from backend.eval.experiment import CONTROL, VideoCandidates, predict
68     from backend.eval.gt_loader import load_gt_intervals
69     from backend.eval.metrics import score
70     from backend.eval.regression import gt_hash
71     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
72
73     Interval = Tuple[float, float]
74
75     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
76     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
77     CURRENT_SCHEMA = 1
78
79     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
80     works
81     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
82     root.
83     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
84
85     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
86     merge_gap,
87     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
88     LITERAL
89     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
90     constant so a
91     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
92     fixture also
93     #: re-states these in its provenance; the replay reads them from there.
94     PINNED_VT = 0.02
95     PINNED_MERGE_GAP = 2.0
96     PINNED_MIN_WINDOW = 2.0
97     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
98     NOTE: this is
99     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
100     ``pinned_proposal``
101     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
102     deliberately
103     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
104     the W1-OFF
105     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
106     wiring.
107     PINNED_MAX_WINDOW = 0.0
108
109     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
110     stated
111     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
112     exactly these.

```

```

99     SHUTTLE_FEATURES: Tuple[str, ...] = (
100         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
101     )
102     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
103         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
104     )
105
106     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
107     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
108     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
109
110     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
111     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
112
113     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
114     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
115     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
116
117     #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
118     #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
119     #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
120     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
121     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
122
123
124     # =====
125     # Synthetic trajectory generation (deterministic ? NO random)
126     # =====
127
128
129     @dataclass(frozen=True)
130     class RallySpec:
131         """One in-flight rally segment: the shuttle oscillates across the net midline.
132
133         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
134         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
135         net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
136         """
137
138         start_frame: int
139         n_frames: int
140         period: float = 30.0
141         amplitude: float = 400.0
142
143
144     @dataclass(frozen=True)
145     class GapSpec:
146         """Dead-time between rallies ? what separates one window from the next.
147
148         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
149         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
end an active run so the next rally forms a distinct window.
150         """
151
152
153         start_frame: int
154         n_frames: int
155         kind: str = "still" # "still" | "occluded"

```

```

156
157
158 @dataclass(frozen=True)
159 class FixtureSpec:
160     """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
161
162     ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
163     "labels.csv". For a synthetic fixture they line up with the rallies by construction,
164     which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
165     field-quality one (§6).
166     """
167
168     slug: str
169     description: str
170     fps: float
171     frame_width: float
172     frame_height: float
173     segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
174     gts: Tuple[Interval, ...]
175     net_midline: float = 640.0
176
177
178     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
179                     ) -> List[Tuple[int, int, float, float]]:
180         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
181         rows: List[Tuple[int, int, float, float]] = []
182         y_mid = frame_height / 2.0
183         for i in range(spec.n_frames):
184             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
185             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
186             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
187             rows.append((spec.start_frame + i, 1, x, y))
188         return rows
189
190
191     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
192                   ) -> List[Tuple[int, int, float, float]]:
193         """Dead-time rows: resting-visible (still) or absent (occluded)."""
194         rows: List[Tuple[int, int, float, float]] = []
195         y_mid = frame_height / 2.0
196         for i in range(spec.n_frames):
197             if spec.kind == "occluded":
198                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
199             elif spec.kind == "still":
200                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
201             else: # pragma: no cover - guarded by builder
202                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still' | 'occluded')")
203         return rows
204
205
206     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
207         """Build the deterministic ``(Frame, Visibility, X, Y)`` rows for a synthetic
scenario."""
208         rows: List[Tuple[int, int, float, float]] = []
209         for seg in spec.segments:
210             if isinstance(seg, RallySpec):
211                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
212             elif isinstance(seg, GapSpec):
213                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
214             else: # pragma: no cover - guarded by dataclass shape
215                 raise TypeError(f"unknown segment type {type(seg).__name__}")
216         rows.sort(key=lambda r: r[0])
217         return rows

```

```

218
219
220 def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
221     """Write trajectory rows as the TrackNet CSV (`Frame,Visibility,X,Y`), LF + 6dp
floats."""
222     lines = ["Frame,Visibility,X,Y"]
223     for frame, vis, x, y in rows:
224         lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
225     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
226
227
228 def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
229     """Write GT intervals as a `start,end` labels CSV (the strict gt_loader format),
LF."""
230     lines = ["start,end"]
231     for s, e in gts:
232         lines.append(f"{_fmt(s)},{_fmt(e)}")
233     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
234
235
236 # =====
237 # Built-in synthetic scenarios (the 3 committed slugs)
238 # =====
239
240
241 def _fr(seconds: float, fps: float) -> int:
242     return int(round(seconds * fps))
243
244
245 def builtin_specs() -> List[FixtureSpec]:
246     """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
247     windowing behaviour (one window / dead-time split / occlusion split)."""
248     fps = 30.0
249     fw, fh = 1280.0, 720.0
250     net = 640.0
251
252     # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
253     single = FixtureSpec(
254         slug="clean_single_rally",
255         description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
256         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
257         segments=(RallySpec(start_frame=0, n_frames=120),),
258         # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
259         gts=((0.0, 4.0),),
260     )
261
262     # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
263     # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
264     r0 = RallySpec(start_frame=0, n_frames=120)
265     gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
266     r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
267     multi = FixtureSpec(
268         slug="multi_rally_with_deadtime",
269         description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
270         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
271         segments=(r0, gap_still, r1),
272         gts=((0.0, 4.0), (7.0, 11.0)),
273     )
274

```

```

275     # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
276     #     visibility break ? proves occlusion (not just low velocity) separates windows.
277     o0 = RallySpec(start_frame=0, n_frames=120)
278     gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
279     o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
280     occ = FixtureSpec(
281         slug="occlusion_break",
282         description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
283         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
284         segments=(o0, gap_occ, o1),
285         gts=((0.0, 4.0), (7.0, 11.0)),
286     )
287
288     return [single, multi, occ]
289
290
291     def _spec_by_slug() -> Dict[str, FixtureSpec]:
292         return {s.slug: s for s in builtin_specs()}
293
294
295     # =====
296     # The shared replay (CONTROL / pure-stdlib path)
297     # =====
298
299
300     def _fmt(x: float) -> str:
301         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
302         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
303         if r == 0.0:
304             r = 0.0
305         return f"{r:.6f}"
306
307
308     def _round(x: float) -> float:
309         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
310         return 0.0 if r == 0.0 else r
311
312
313     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
314         """The windowing proposal tuple ``(vt, mg, mwd)`, read from the fixture provenance
when present
315         (the pinned params travel with the fixture), else the module-level pinned defaults.
NEVER
316         ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
read here ?
317         ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
path; the
318         pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
PINNED_MAX_WINDOW)."""
319         if prov and "windowing" in prov:
320             w = prov["windowing"]
321             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
322         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
323
324
325     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
326         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
327
328         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
329         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:

```



```

330
331         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
332         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
333
334     Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
335     control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
336     merges overlaps; NO numpy is touched.
337     ""
338     fdir = _resolve_dir(slug_or_dir)
339     slug = fdir.name
340     traj_csv = fdir / "trajectory.csv"
341     labels_csv = fdir / "labels.csv"
342     prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
343
344     fps, fw = _fps_fw(prov, slug)
345     iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
346     proposal = _pinned_proposal(prov)
347
348     intervals, feature_rows = distill_local.build_candidates(
349         str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
350
351     # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
352     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354     features: Dict[str, List[float]] = {
355         name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356     }
357     gts = load_gt_intervals(str(labels_csv), strict=True)
358     vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360     preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361     sc = score(preds, gts, iou)
362
363     candidates: List[Dict[str, Any]] = []
364     for j, (s, e) in enumerate(intervals):
365         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
366         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
367         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
368         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
369
370     return {
371         "slug": slug,
372         "schema_version": CURRENT_SCHEMA,
373         "iou": _round(iou),
374         "candidates": candidates,
375         "control_segments": [[_round(s), _round(e)] for s, e in preds],
376         "score": {
377             "iou_threshold": _round(sc.iou_threshold),
378             "num_pred": int(sc.num_pred),
379             "num_gt": int(sc.num_gt),
380             "true_positives": int(sc.true_positives),
381             "false_positives": int(sc.false_positives),
382             "false_negatives": int(sc.false_negatives),
383             "precision": _round(sc.precision),
384             "recall": _round(sc.recall),

```

```

385         "f1": _round(sc.f1),
386         "mean_iou": _round(sc.mean_iou),
387         "mean_start_error": _round(sc.mean_start_error),
388         "mean_end_error": _round(sc.mean_end_error),
389     },
390     "gt_hash": gt_hash(gts),
391 }
392
393
394 # =====
395 # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
396 # =====
397
398
399 def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
400                 ) -> Dict[str, Any]:
401     """The committed provenance block ? pins the windowing + privacy/licensing tags
402     (§4c, §5)."""
403     desc = ""
404     if isinstance(spec_or_dir, FixtureSpec):
405         desc = spec_or_dir.description
406     return {
407         "slug": slug,
408         "schema_version": CURRENT_SCHEMA,
409         "kind": "synthetic",
410         "license": "synthetic",
411         "minors_free": True,
412         "description": desc,
413         "fps": _round(fps),
414         "frame_width": _round(fw),
415         "iou": _round(iou),
416         "windowing": {
417             "vt": PINNED_VT,
418             "mg": PINNED_MERGE_GAP,
419             "mwd": PINNED_MIN_WINDOW,
420             "max_win": PINNED_MAX_WINDOW,
421         },
422         "paths": {
423             "trajectory": "trajectory.csv",
424             "labels": "labels.csv",
425             "expected": "expected.json",
426         },
427         "note": (
428             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
429             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
430 (§6).",
431     ),
432 }
433
434 def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
435                   iou: float = 0.5) -> Dict[str, Any]:
436     """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
437     COMMITTED
438     ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
439     construction).
440
441     Idempotent: rewrites the same bytes given the same committed inputs. Requires the
442     trajectory
443     + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
444     """
445     fdir = fixtures_dir / slug
446     traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
447     if not traj_csv.is_file() or not labels_csv.is_file():
448         raise FileNotFoundError(

```

```

445         f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
446
447     # Provenance first (it pins the windowing the replay reads), then replay ? expected.
448     spec = _spec_by_slug().get(slug)
449     fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
450     prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
451     _write_json(fdir / "provenance.json", prov)
452
453     expected = replay_fixture(fdir)
454     _write_json(fdir / "expected.json", expected)
455     return expected
456
457
458     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
459         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
460         provenance.json, and (re)write the manifest. Returns the slugs frozen."""
461         specs = builtin_specs()
462         fixtures_dir.mkdir(parents=True, exist_ok=True)
463         slugs: List[str] = []
464         for spec in specs:
465             fdir = fixtures_dir / spec.slug
466             fdir.mkdir(parents=True, exist_ok=True)
467             rows = make_synthetic_trajectory(spec)
468             write_trajectory_csv(rows, fdir / "trajectory.csv")
469             write_labels_csv(spec.gts, fdir / "labels.csv")
470             freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
471             slugs.append(spec.slug)
472         _write_manifest(specs, fixtures_dir)
473         return slugs
474
475
476     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
477                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
478         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
479         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
regression; the
480         mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
481         if all_slugs:
482             targets = [d.name for d in sorted(fixtures_dir.iterdir())
483                       if d.is_dir() and (d / "trajectory.csv").is_file()]
484         elif slug:
485             targets = [slug]
486         else:
487             raise ValueError("refresh_snapshot needs --slug S or --all")
488         for s in targets:
489             freeze_fixture(s, fixtures_dir=fixtures_dir)
490         return targets
491
492
493     # =====
494     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
495     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
496     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
497     # =====
498
499
500     class RefusalError(ValueError):
501         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
502
503         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch

```

```

it) that
504     names *why* the unsafe public-fixture path was refused. The refusal is the code-
enforced
505     provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
minors-excluded
506     / license flags are human-asserted, but the gate makes the unsafe path impossible to
reach
507     *silently* ? a fixture cannot be written unless all three are affirmatively set.
508     ""
509
510
511     def _new_surrogate_slug() -> str:
512         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
513
514         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
random,
515         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
over a
516         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
517         ""
518         return f"fx_{os.urandom(8).hex()}"
519
520
521     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
path token
522     #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
regardless), so it is
523     #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
would let
524     #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
(red-team #4).
525     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
526         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
527         "provenance", "provenance.json",
528     )
529
530
531     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
532         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.
533
534         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
535         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
536         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
537         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
538         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
539         ""
540         low = text.lower()
541         for key in FORBIDDEN_KEYS:
542             if key in low:
543                 raise RefusalError(
544                     f"{where}: forbidden key substring {key!r} found in written output ? "
545                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
546                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
547         m = _MD5_RE.search(text)
548         if m:
549             raise RefusalError(
550                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is

```

```

the "
551         f"video_id shape and an attribution back-channel; never write it ($4c/$5).")
552     for sp in source_paths:
553         if not sp:
554             continue
555         p = Path(sp)
556         for token in (sp, p.name, p.stem):
557             t = (token or "").strip().lower()
558             # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
559             # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
560             # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
561             # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
562             # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
563             if not t or t in _SCHEMA_FILENAME_LITERALS:
564                 continue
565             if t in low:
566                 raise RefusalError(
567                     f"{where}: source-path token {token!r} found in written output ? the
source "
568                     f"filename/path must never be persisted ($4c metadata strip).")
569
570
571     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
572                               ) -> List[Tuple[int, int, float, float]]:
573         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
574
575         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
576         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
577         """
578         rows: List[Tuple[int, int, float, float]] = []
579         for p in points:
580             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
581         rows.sort(key=lambda r: r[0])
582         return rows
583
584
585     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
586         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant).
587
588         ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
Each label
589         endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
of two exact
590         6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
591         ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
reset score
592         delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
``freeze_real_fixture``
593         for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
second
594         offset (the grid, not a real metric divergence)."""
595         return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]
596
597
598     def freeze_real_fixture(
599         label: str,

```

```

600     trajectory_csv_path: Any,
601     labels_csv_path: Any,
602     *,
603     license: str,
604     minors_free: bool,
605     owned: bool,
606     fps: float,
607     frame_width: float,
608     fixtures_dir: Path = FIXTURES_DIR,
609     slug: Optional[str] = None,
610     time_origin_reset: bool = True,
611     source_note: str = "",
612 ) -> Dict[str, Any]:
613     """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
614     Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
row P1.)
615
616     The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
617     ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string,
618     AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text
identifier like a
619     player/event name is refused, red-team #1). The committed bytes carry NO filename /
video_id /
620     match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
621     shuttle-only trajectory (time-origin reset by default), the reset labels, and a
622     ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
623     ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.
624
625     ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
626     back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
627     contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
628     committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
629     retained for CLI compatibility / future logging.
630
631     NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
632     only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
633     attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
634     owner-asserted Fork-A / minors / biometric-exclusion flags.
635
636     The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
637     after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
638     (red-team #3).
639
640     Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
641     """
642     # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
643     reasons: List[str] = []
644     if minors_free is not True:

```

```

645         reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
646                         "of a minor is prohibited ? §3, guardrail 'exclude minors')")
647     if owned is not True:
648         reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
649                         "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
650     if not (isinstance(license, str) and license.strip()):
651         reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
652                         "'provenance hard-fail gate')")
653     # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
654     # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
655     if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
656         reasons.append(
657             f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}
(lowercase "
658             f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier
(player/event "
659             f"name) is an attribution leak; got {label!r} (§4c metadata strip, red-team
#1)")
660     if reasons:
661         raise RefusalError(
662             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
663             "cure provenance/licensing; this gate sits behind counsel sign-off,
§6/P2):\n - "
664             + "\n - ".join(reasons))
665
666     traj_path = Path(str(trajjectory_csv_path))
667     labels_path = Path(str(labels_csv_path))
668
669     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
670     out_slug = slug if slug is not None else _new_surrogate_slug()
671
672     # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
----
673     points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
674     if not points:
675         raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
676     gts = load_gt_intervals(str(labels_path), strict=True)
677
678     # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, §4c) --
679     # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
680     # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
681     # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
682     # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
683     # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
684     # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
685     #
686     # HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while
labels shift
687     # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),

```

```

688         # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
689         # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
690         # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).
691         if time_origin_reset:
692             t0_frame = min(int(p.frame) for p in points)
693             t0_sec = _round(t0_frame / float(fps))
694         else:
695             t0_frame, t0_sec = 0, 0.0
696         rows = _reset_trajectory_rows(points, t0_frame)
697         reset_gts = _reset_labels(gts, t0_sec)
698
699         # --- (d2) negative-label guard BEFORE any write (red-team #3a)
-----
700         # A reset label that starts < 0 means the label timeline predates the earliest
trajectory
701         # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
702         # start). Refuse loudly up-front so the partial-write path is never even reached.
703         neg = [(s, e) for (s, e) in reset_gts if s < 0]
704         if neg:
705             raise RefusalError(
706                 f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "
707                 f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
708                 f"earliest label must not precede the earliest trajectory frame ($4c time-
origin reset).")
709
710         iou = 0.5
711         # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
712         # it never reaches the committed bytes (an identifying note is an attribution back-
channel
713         # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
714         prov = {
715             "slug": out_slug,
716             "schema_version": CURRENT_SCHEMA,
717             "kind": "cleared_real",
718             "label": str(label),
719             "license": license,
720             "minors_free": True,
721             "owned": True,
722             "fps": _round(fps),
723             "frame_width": _round(frame_width),
724             "iou": _round(iou),
725             "time_origin_reset": bool(time_origin_reset),
726             "windowing": {
727                 "vt": PINNED_VT,
728                 "mg": PINNED_MERGE_GAP,
729                 "mwd": PINNED_MIN_WINDOW,
730                 "max_win": PINNED_MAX_WINDOW,
731             },
732             "paths": {
733                 "trajectory": "trajectory.csv",
734                 "labels": "labels.csv",
735                 "expected": "expected.json",
736             },
737             "gt_hash": gt_hash(reset_gts),
738             "note": (
739                 "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"

```



```

740         "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
741         "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
742         "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2). "
743     ),
744 }
745
746 # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
-----
747 # Everything is written under a sibling temp dir; the final slug dir is created by
an atomic
748 # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
leaves NO
749 # partial fixture dir behind. source_note (if any) is scanned defensively but never
written.
750 fdir = fixtures_dir / out_slug
751 fixtures_dir.mkdir(parents=True, exist_ok=True)
752 if fdir.exists():
753     raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
(choose a new slug)")
754 tmp_parent = Path(tempfile.mkdtemp(prefix=f"{out_slug}.tmp.",
dir=str(fixtures_dir)))
755 tmp_dir = tmp_parent / out_slug
756 tmp_dir.mkdir(parents=True, exist_ok=True)
757 try:
758     write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
759     write_labels_csv(reset_gts, tmp_dir / "labels.csv")
760     _write_json(tmp_dir / "provenance.json", prov)
761
762     expected = replay_fixture(tmp_dir) # REUSE M1 ? correct-by-construction
snapshot
763     _write_json(tmp_dir / "expected.json", expected)
764
765     # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
source_note) ---
766     source_paths = [str(traj_path), str(labels_path)]
767     _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
768                     where=f"{out_slug}/expected.json", source_paths=source_paths)
769     _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
770                     where=f"{out_slug}/provenance.json", source_paths=source_paths)
771     # source_note is never persisted, but a leak in it is still a contract violation
? scan it.
772     if source_note:
773         _scan_forbidden(str(source_note),
774                         where=f"{out_slug}/source_note(not-persisted)",
source_paths=source_paths)
775
776     # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
-----
777     os.replace(str(tmp_dir), str(fdir))
778 finally:
779     # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
partial dir).
780     shutil.rmtree(str(tmp_parent), ignore_errors=True)
781
782     # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
783     _append_manifest_entry(prov, fixtures_dir)
784     return expected
785
786
787 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
788     """A manifest row for a single fixture, derived from its provenance block."""

```

```

789     slug = prov["slug"]
790     return {
791         "slug": slug,
792         "fps": _round(float(prov["fps"])),
793         "frame_width": _round(float(prov["frame_width"])),
794         "windowing": dict(prov.get("windowing", {})),
795         "kind": prov.get("kind", "synthetic"),
796         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
797         "paths": {
798             "dir": slug,
799             "trajectory": f"{slug}/trajectory.csv",
800             "labels": f"{slug}/labels.csv",
801             "expected": f"{slug}/expected.json",
802             "provenance": f"{slug}/provenance.json",
803         },
804     }
805
806
807     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
808         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
809         synthetic)."""
810         entries = load_manifest(fixtures_dir)
811         entries = [e for e in entries if e.get("slug") != prov["slug"]]
812         entries.append(_manifest_entry_for(prov))
813         entries.sort(key=lambda e: str(e.get("slug")))
814         _write_json(fixtures_dir / "manifest.json", entries)
815
816     # =====
817     # Manifest + loaders
818     # =====
819
820
821     def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
822         entries = [{
823             "slug": spec.slug,
824             "fps": _round(spec.fps),
825             "frame_width": _round(spec.frame_width),
826             "windowing": {
827                 "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
828                 "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
829             },
830             "kind": "synthetic",
831             "schema_version": CURRENT_SCHEMA,
832             "paths": {
833                 "dir": spec.slug,
834                 "trajectory": f"{spec.slug}/trajectory.csv",
835                 "labels": f"{spec.slug}/labels.csv",
836                 "expected": f"{spec.slug}/expected.json",
837                 "provenance": f"{spec.slug}/provenance.json",
838             },
839         } for spec in specs]
840         _write_json(fixtures_dir / "manifest.json", entries)
841
842
843     def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
844         """Load the fixtures manifest (returns [] if absent)."""
845         path = fixtures_dir / "manifest.json"
846         if not path.is_file():
847             return []
848         data = _read_json(path)
849         if not isinstance(data, list): # pragma: no cover - corrupt manifest
850             raise ValueError(f"{path}: manifest must be a JSON array")
851         return data
852

```

```

853
854     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
855         """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
856         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
857         ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
858         bump in one is never masked by an in-range value in the other (red-team #7)."""
859         fdir = fixtures_dir / slug
860         prov = _read_json(fdir / "provenance.json")
861         expected = _read_json(fdir / "expected.json")
862         for which, blob in (("expected", expected), ("provenance", prov)):
863             sv = int(blob.get("schema_version", 0))
864             if sv > CURRENT_SCHEMA:
865                 raise ValueError(
866                     f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "
867                     f"upgrade the harness")
868         return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
869
870
871     # =====
872     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
873     # =====
874
875
876     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
877         """Return a list of human-readable mismatch messages (empty == match).
878
879         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
880         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
881         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
882         safe (the repo runs with core.autocrlf=true).
883
884         Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
885         per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
886         a fixed-field loop alone would ignore an added field (red-team #5)."""
887         out: List[str] = []
888
889         def exact(path: str, a: Any, b: Any) -> None:
890             if a != b:
891                 out.append(f"{path}: {a!r} != {b!r} (exact)")
892
893         def approx(path: str, a: Any, b: Any) -> None:
894             try:
895                 if abs(float(a) - float(b)) > _FLOAT_TOL:
896                     out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
897             except (TypeError, ValueError):
898                 out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
899
900         # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
901         if set(recomputed) != set(committed):
902             out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
903
904         exact("slug", recomputed.get("slug"), committed.get("slug"))
905         exact("schema_version", recomputed.get("schema_version"),

```

```

committed.get("schema_version"))
906     exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
907     approx("iou", recomputed.get("iou"), committed.get("iou"))
908
909     rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
910     if len(rc) != len(cc):
911         out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
912     else:
913         for i, (r, c) in enumerate(zip(rc, cc)):
914             approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
915             approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
916             rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
917             if set(rs) != set(cs):
918                 out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
919             else:
920                 for k in rs:
921                     if k == "net_crossings":
922                         exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
923                     else:
924                         approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
925
926     rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
927     if len(rseg) != len(cseg):
928         out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
929     else:
930         for i, (r, c) in enumerate(zip(rseg, cseg)):
931             approx(f"control_segments[{i}][0]", r[0], c[0])
932             approx(f"control_segments[{i}][1]", r[1], c[1])
933
934     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
935     # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
936     if set(rsc) != set(csc):
937         out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
938     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
939     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
"mean_start_error", "mean_end_error")
940
941     for k in int_fields:
942         exact(f"score.{k}", rsc.get(k), csc.get(k))
943     for k in float_fields:
944         approx(f"score.{k}", rsc.get(k), csc.get(k))
945
946     return out
947
948
949 # =====
950 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
951 # =====
952
953
954 def _write_json(path: Path, obj: Any) -> None:
955     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
956     path.write_text(text + "\n", encoding="utf-8", newline="\n")
957
958
959 def _read_json(path: Path) -> Any:
960     with open(path, encoding="utf-8") as f:
961         return json.load(f)
962
963
964 def _resolve_dir(slug_or_dir: Any) -> Path:
965     if isinstance(slug_or_dir, Path):

```

```

966         return slug_or_dir
967     p = Path(str(slug_or_dir))
968     if p.is_dir():
969         return p
970     return FIXTURES_DIR / str(slug_or_dir)
971
972
973 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
974     if prov and "fps" in prov and "frame_width" in prov:
975         return float(prov["fps"]), float(prov["frame_width"])
976     spec = _spec_by_slug().get(slug)
977     if spec is not None:
978         return spec.fps, spec.frame_width
979     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
980
981
982 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
983     prov_path = fdir / "provenance.json"
984     prov = _read_json(prov_path) if prov_path.is_file() else None
985     return _fps_fw(prov, slug)
986
987
988 # =====
989 # CLI
990 # =====
991
992
993 def _cmd_check(fixtures_dir: Path) -> int:
994     manifest = load_manifest(fixtures_dir)
995     if not manifest:
996         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
997         return 1
998     failures = 0
999     for entry in manifest:
1000         slug = entry["slug"]
1001         committed = _read_json(fixtures_dir / slug / "expected.json")
1002         recomputed = replay_fixture(fixtures_dir / slug)
1003         mismatches = compare_expected(recomputed, committed)
1004         if mismatches:
1005             failures += 1
1006             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
f"(characterization = behaviour-locking, NOT correctness ? $6):")
1007             for m in mismatches:
1008                 print(f"    - {m}")
1009         else:
1010             sc = recomputed["score"]
1011             print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
1012     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
"
1013     f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
1014     return 0 if failures == 0 else 1
1015
1016
1017
1018
1019 def main(argv: Optional[Sequence[str]] = None) -> int:
1020     ap = argparse.ArgumentParser(
1021         description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
harness.")
1022     g = ap.add_mutually_exclusive_group(required=True)
1023     g.add_argument("--freeze-synthetic", action="store_true",
1024         help="generate synthetic trajectory+labels for all built-in
scenarios, then "
1025         "freeze expected.json+provenance.json + write manifest.json")

```

```

1026         g.add_argument("--refresh-snapshot", action="store_true",
1027                         help="re-freeze expected.json from COMMITTED trajectory+labels (no
regeneration)")
1028         g.add_argument("--check", action="store_true",
1029                         help="replay all fixtures + parse-compare vs committed expected.json
(exit 0/1)")
1030         g.add_argument("--freeze-real", action="store_true",
1031                         help="(P1) de-attribute an OWNED real trajectory+labels into a
committable Tier-0 "
1032                             "fixture; REFUSED without --owned --minors-free and a non-blank
--license")
1033         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
slug for "
1034                             "--freeze-real (default: a random fx_<hex> id)")
1035         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
snapshot")
1036         ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
(review the diff!)")
1037         # --freeze-real (P1 / M2 refusal gate) options:
1038         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
1039         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
1040         ap.add_argument("--license", dest="license_", default=None,
1041                         help="--freeze-real) the source license tag (required, non-blank)")
1042         ap.add_argument("--owned", action="store_true",
1043                         help="--freeze-real) assert owner-recorded Fork-A source
(required)")
1044         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1045                         help="--freeze-real) assert the source contains NO minors
(required)")
1046         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
1047         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
1048         ap.add_argument("--label", default="cleared_real",
1049                         help="--freeze-real) non-identifying scenario label ? STRICT slug
only "
1050                             "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
1051         ap.add_argument("--source-note", dest="source_note", default="",
1052                         help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
1053                             "attribution back-channel, so it never reaches the committed
provenance "
1054                             "(scanned defensively, then dropped). Retained for CLI
compatibility.")
1055         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1056                         help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
1057         ap.set_defaults(time_origin_reset=True)
1058         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1059                         help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
1060         args = ap.parse_args(list(argv) if argv is not None else None)
1061
1062         fixtures_dir: Path = args.fixtures_dir
1063
1064         if args.freeze_synthetic:
1065             slugs = freeze_synthetic(fixtures_dir)
1066             print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
''.join(slugs)}")
1067             print(f" -> {fixtures_dir}")
1068             return 0
1069
1070         if args.refresh_snapshot:

```

```

1071         if not args.slug and not args.all:
1072             ap.error("--refresh-snapshot requires --slug S or --all")
1073         # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
1074         real
1075         # regression, so the deliberate, recorded reason is a required arg, not a soft
1076         warning.
1077         if not (args.reason and args.reason.strip()):
1078             ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
1079             stamp a real \"
1080             \"regression; record the deliberate reason and review the visible
1081             diff ($6).\"")
1082             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
1083             fixtures_dir=fixtures_dir)
1084             print(f"[golden-fixtures] refreshed snapshot for: {'', '.join(targets)}\"
1085             f\" (reason: {args.reason})\"")
1086             return 0
1087
1088         if args.freeze_real:
1089             missing = [n for n, v in (("--trajectory", args.trajectory),
1090             ("--labels", args.labels),
1091             ("--fps", args.fps),
1092             ("--frame-width", args.frame_width)) if v is None]
1093             if missing:
1094                 ap.error("--freeze-real requires \" + \"\", \".join(missing))
1095             try:
1096                 expected = freeze_real_fixture(
1097                     args.label, args.trajectory, args.labels,
1098                     license=args.license_ if args.license_ is not None else "",
1099                     minors_free=args.minors_free, owned=args.owned,
1100                     fps=args.fps, frame_width=args.frame_width,
1101                     fixtures_dir=fixtures_dir, slug=args.slug,
1102                     time_origin_reset=args.time_origin_reset, source_note=args.source_note)
1103             except RefusalError as e:
1104                 # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
1105                 zero.
1106                 print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}\"",
1107                 file=sys.stderr)
1108                 return 2
1109             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'\"
1110             \"
1111             f\"(kind=cleared_real) -> {fixtures_dir / expected['slug']}\"")
1112             print(f" $6: behaviour-locking, NOT correctness; non-attributable only relative
1113             to the \"
1114             \"UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
1115             (P2).\"")
1116             return 0
1117
1118         # --check
1119         return _cmd_check(fixtures_dir)
1120
1121         if __name__ == "__main__":
1122             raise SystemExit(main())
1123
1124

```

7. assistant (2026-06-18T07:54:48.395Z)

Now let me read the other three cluster files.

8. user (2026-06-18T07:54:49.849Z)

```

1      """"Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3      A *systematic, repeatable* version of the one-off ablation
(``scratch/gate_ab_ablation.py``):

```

```

4      run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
per
5      signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
bootstrap
6      95% CI + paired sign test + the C4 over-segmentation ratio. See
``scratch/ABLATION_FRAMEWORK_PLAN.md``
7      (the design) and ``docs/ABLATION_LAYER.md`` (the shipped note).
8
9      **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
10     ``experiment.compare`` + ``eval_stats`` engine, exactly as ``compare`` was a driver over
11     ``metrics``/``calibration``. For each signal ``s`` the marginal contribution is:
12
13     ?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop, ?F1
primary
14
15     operationally ``experiment.compare(videos, variant_a=without_s, variant_b=full)`` ? A is
the
16     ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
paired ?F1
17     with bootstrap CI + exact sign test, LOVO-honest for learned variants.
18
19     Two ablation granularities fall out of one inventory:
20
21     - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
22     - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
owner's
23     "is this old SIGNAL still relevant?" question at cue granularity).
24
25     A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
weights) or
26     a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
by
27     dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
columns AND/OR
28     zeroes the gate knob from ``cfg.full``.
29
30     Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; §6 of the
plan):
31     - never a bare mean ? every row ships a CI + the exact sign test;
32     - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
sign test
33       agreeing** (the ``per_user_gate`` bar, in reverse);
34     - **structural signals are exempt from auto-demotion** ? a guard whose value is in
failure modes
35     the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
is NEVER
36     flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
? honestly
37     but tags ``structural_protected``;
38     - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39     MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41     Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42     writes one ``<name>_golden_features.json`` per video). CLI::
43
44     python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45     """
46     from __future__ import annotations
47
48     import argparse
49     import glob
50     import json

```



```

51     import logging
52     import os
53     from dataclasses import dataclass, field
54     from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56     from backend.eval import experiment
57     from backend.eval.experiment import Variant, VideoCandidates, _is_fusion_schema
58
59     logger = logging.getLogger(__name__)
60
61     # --- The signal inventory (the things the runner ablates), grouped by producer.
62     -----
63     # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
64     # (person), audio_features (audio). The shuttle group's velocity block + duration are
65     the
66     # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
67     knob.
68     SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
69     "active_ratio", "net_crossings")
70     PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
71     "mean_player_motion",
72     "in_court_ratio", "shuttle_player_colocation")
73     AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
74     "audio_hit_strength_mean")
75
76     # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
77     EARNNS_KEEP = "earns_keep" # CI clears 0 AND sign test agrees ? removing it
78     hurts
79     SUGGESTIVE = "suggestive" # point estimate leans positive but CI spans 0
80     INERT = "inert" # wash this run (?0, CI spans 0) ? candidate,
81     not proof
82     STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no
83     lift"
84
85     #: Human-readable one-liners for each verdict (used in the PDF + table legend).
86     VERDICT_LABEL = {
87         EARNNS_KEEP: "clears-bar",
88         SUGGESTIVE: "suggestive",
89         INERT: "inert",
90         STRUCTURAL_PROTECTED: "structural-protected",
91     }
92
93     @dataclass(frozen=True)
94     class Signal:
95         """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
96
97         - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
98         - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a
99         whole group.
100         - ``columns`` ? the persisted feature columns this signal contributes (may be empty
101         for a
102         pure gate signal like Gate-A whose column is shared with the shuttle group).
103         - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
104         zeroing it on
105         the full variant rather than dropping a column.
106         - ``structural`` ? a guard whose value is in failure modes the golden corpus may
107         lack; NEVER
108         flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
109         ``structural=True``).
110         """
111         name: str
112         group: str
113         columns: Tuple[str, ...] = ()

```

```

103     gate_knob: Optional[str] = None
104     structural: bool = False
105
106
107     @dataclass
108     class AblationConfig:
109         """How to run an ablation over a corpus.
110
111         - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112         the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
113         ``feature_names`` = all columns and the gates ON.
114         - ``signals`` ? the inventory to ablate.
115         - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
116         - ``iou`` ? temporal-IoU match threshold.
117         - ``honest`` ? LOVO for learned variants (the number to trust).
118         - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119         - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
120         """
121
122         full: Variant
123         signals: List[Signal]
124         granularity: str = "group"
125         iou: float = 0.5
126         honest: bool = True
127         seed: int = 0
128         p_threshold: float = 0.05
129
130
131     @dataclass
132     class AblationRow:
133         """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a
verdict)."""
134
135         signal: str
136         group: str
137         structural: bool
138         dropped_columns: List[str]
139         gate_knob: Optional[str]
140         n: int
141         delta_f1: float
142         ci_low: float
143         ci_high: float
144         ci_excludes_zero: bool
145         sign_wins: int
146         sign_losses: int
147         sign_p_value: float
148         f1_full: float
149         f1_ablated: float
150         seg_ratio_full: float
151         seg_ratio_ablated: float
152         verdict: str
153         structural_protected: bool
154
155         def as_dict(self) -> Dict[str, Any]:
156             return {
157                 "signal": self.signal,
158                 "group": self.group,
159                 "structural": self.structural,
160                 "dropped_columns": list(self.dropped_columns),
161                 "gate_knob": self.gate_knob,
162                 "n": self.n,
163                 "delta_f1": self.delta_f1,

```

```

164         "ci_low": self.ci_low,
165         "ci_high": self.ci_high,
166         "ci_excludes_zero": self.ci_excludes_zero,
167         "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                       "p_value": self.sign_p_value},
169         "f1_full": self.f1_full,
170         "f1_ablated": self.f1_ablated,
171         "segment_count_ratio_full": self.seg_ratio_full,
172         "segment_count_ratio_ablated": self.seg_ratio_ablated,
173         "verdict": self.verdict,
174         "structural_protected": self.structural_protected,
175     }
176
177
178     @dataclass
179     class AblationReport:
180         """All signals' rows + the run's provenance ? a structured, JSON-able ablation
181         result."""
182         granularity: str
183         iou: float
184         num_videos: int
185         honest_lovo: bool
186         full_set_signals: List[str]
187         full_spec_hash: str
188         label_set_hash: str
189         rows: List[AblationRow] = field(default_factory=list)
190
191         def sorted_rows(self) -> List[AblationRow]:
192             """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
193             order)."""
194             return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
195
196         def as_dict(self) -> Dict[str, Any]:
197             return {
198                 "granularity": self.granularity,
199                 "iou": self.iou,
200                 "num_videos": self.num_videos,
201                 "honest_lovo": self.honest_lovo,
202                 "full_set_signals": list(self.full_set_signals),
203                 "full_spec_hash": self.full_spec_hash,
204                 "label_set_hash": self.label_set_hash,
205                 "rows": [r.as_dict() for r in self.sorted_rows()],
206             }
207
208     # ----- the driver
209
210
211     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
212     Variant:
213         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
214         its
215         ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
216         never
217         mutates ``full``."""
218         cols = set(columns)
219         fn = full.feature_names
220         ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
221         overrides: Dict[str, Any] = {"feature_names": ablated_fn}
222         if gate_knob == "min_crossings":
223             overrides["min_crossings"] = 0
224         elif gate_knob == "min_players":
225             overrides["min_players"] = 0
226         elif gate_knob is not None:

```

```

224         raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
225         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227                                           "name": f"?{dropped_label}")
228
229
230     def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
231                  sign_p: float, p_threshold: float) -> Tuple[str, bool]:
232         """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
233
234         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
235         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
236         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
237         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
238         if structural:
239             return STRUCTURAL_PROTECTED, True
240         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241         if ci_excludes_zero and sign_agrees:
242             return EARNES_KEEP, False
243         if sign_wins > sign_losses:          # leans toward "removal hurts" but evidence is
thin
244             return SUGGESTIVE, False
245         return INERT, False
246
247
248     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249                  signal: Signal) -> AblationRow:
250         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251
252         Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253         variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
?(s).
254         Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
255         come for free."""
256         ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257         res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                                honest_stats=True)
259         # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
260         # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261         from backend.eval.eval_stats import paired_delta_ci
262         by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}
263         by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264         vids = [p["video"] for p in res["variant_a"]["per_video"]]
265         pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
266         st = pd["sign_test"]
267         sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
268         sign_p = float(st["p_value"])
269         verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
sign_wins, sign_losses, sign_p, cfg.p_threshold)
270
271
272         seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
273         seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
274
275         return AblationRow(
276             signal=signal.name,
277             group=signal.group,

```

```

278         structural=signal.structural,
279         dropped_columns=list(signal.columns),
280         gate_knob=signal.gate_knob,
281         n=int(pd["n"]),
282         delta_f1=float(pd["mean_delta"]),
283         ci_low=float(pd["ci_low"]),
284         ci_high=float(pd["ci_high"]),
285         ci_excludes_zero=bool(pd["ci_excludes_zero"]),
286         sign_wins=sign_wins,
287         sign_losses=sign_losses,
288         sign_p_value=sign_p,
289         f1_full=float(res["variant_b"]["aggregate"]["f1"]),
290         f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
291         seg_ratio_full=float(seg_full),
292         seg_ratio_ablated=float(seg_abl),
293         verdict=verdict,
294         structural_protected=protected,
295     )
296
297
298     def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
299         """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
300         OR the gate knob, OR the structural flag. First-seen group order is preserved."""
301         order: List[str] = []
302         by_group: Dict[str, List[Signal]] = {}
303         for s in signals:
304             if s.group not in by_group:
305                 order.append(s.group)
306                 by_group[s.group] = []
307             by_group[s.group].append(s)
308         out: List[Signal] = []
309         for g in order:
310             members = by_group[g]
311             cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
312             knob = next((m.gate_knob for m in members if m.gate_knob), None)
313             structural = any(m.structural for m in members)
314             out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
315         return out
316
317
318     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
319         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).
320
321         A pure driver: each row is one ``experiment.compare(without_s, full)``.
``granularity="group"``
322         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
323         time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
324         already persisted)."""
325         if len(videos) < 1:
326             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
327         if cfg.granularity not in ("signal", "group"):
328             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
329
330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]

```

```

332
333     # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334     full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335     from backend.eval.regression import gt_hash
336     all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337     return AblationReport(
338         granularity=cfg.granularity,
339         iou=cfg.iou,
340         num_videos=len(videos),
341         honest_lovo=bool(full_eval["honest_lovo"]),
342         full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343         full_spec_hash=cfg.full.spec_hash(),
344         label_set_hash=gt_hash(all_gts),
345         rows=rows,
346     )
347
348
349     # ----- default
inventories
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces
the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test."""
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``)."""
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381

```

```

382
383 # ----- corpus
I/O
384
385
386 def load_videos(features_dir: str) -> List[VideoCandidates]:
387     """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a
``VideoCandidates``
388     (intervals + shuttle + person + audio columns + golden gts). Audio columns are
optional
389     (older JSONs predate the audio augment) ? a missing block defaults those columns to
0.0 so
390     the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.
391
392     Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
393     R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus
loads the
394     ablatable subset cleanly (see ``_is_fusion_schema``)."""
395     videos: List[VideoCandidates] = []
396     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
397         with open(path, encoding="utf-8") as f:
398             d = json.load(f)
399             cands = d["candidates"]
400             if not _is_fusion_schema(cands):
401                 logger.info("ablation.load_videos: skipping %s ? windowing-only schema "
402                             "(no shuttle/person cue dicts), not an ablatable fusion-feature
file",
403                             os.path.basename(path))
404                 continue
405             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
406             features: Dict[str, List[float]] = {}
407             for fn in SHUTTLE_COLUMNS:
408                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
409             for fn in PERSON_COLUMNS:
410                 features[fn] = [float(c["person"][fn]) for c in cands]
411             for fn in AUDIO_COLUMNS:
412                 features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
413             gts = [(float(a), float(b)) for a, b in d["gts"]]
414             videos.append(VideoCandidates(name=d["name"], intervals=intervals,
415                                           features=features, gts=gts))
416     return videos
417
418
419 def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
420     """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
421     by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
422     (distinct from the static Gate-A litmus)."""
423     cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
424     return experiment.Variant(name="full_learned", feature_names=cols,
425                               threshold=threshold, min_crossings=min_crossings)
426
427
428 # ----- table
rendering
429
430
431 def format_table(report: AblationReport) -> str:
432     """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
ranking)."""
433     rows = report.sorted_rows()
434     header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
435              f" · n={report.num_videos} · IoU {report.iou}"

```

```

436         f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
437         f" . {report.granularity.upper()}")
438     cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
439     widths = [18, 17, 18, 14, 5, 22]
440
441     def _fmt_row(cells: Sequence[str]) -> str:
442         return " ".join(c.ljust(w) for c, w in zip(cells, widths))
443
444     lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
445     for r in rows:
446         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
447         sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
448         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
449         lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
450                                "yes" if r.ci_excludes_zero else "no", verdict]))
451     return "\n".join(lines)
452
453
454     # ----- PDF
455 export
456
457     def export_pdf(report: AblationReport, path: str) -> str:
458         """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
459         marginal
460         contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
461         structural-protected). Reuses the muted booktabs aesthetic of
462         ``scratch/quality_report.py``.
463
464         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
465         GPU)."""
466         from reportlab.lib import colors
467         from reportlab.lib.enums import TA_CENTER, TA_LEFT
468         from reportlab.lib.pagesizes import A4
469         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
470         from reportlab.lib.units import cm
471         from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
472         TableStyle)
473
474         navy = colors.HexColor("#1F3A5F")
475         black = colors.HexColor("#000000")
476         darkgrey = colors.HexColor("#333333")
477         light = colors.HexColor("#F0F0F0")
478         hairline = colors.HexColor("#BFBFBF")
479         row_alt = colors.HexColor("#F7F7F7")
480
481         base = getSampleStyleSheet()["Normal"]
482         title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
483         fontSize=16,
484         leading=20, alignment=TA_CENTER, textColor=navy,
485         spaceAfter=4)
486         subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
487         fontSize=9.5,
488         leading=13, alignment=TA_CENTER, textColor=darkgrey,
489         spaceAfter=10)
490         h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
491         leading=15,
492         textColor=navy, spaceBefore=10, spaceAfter=4)
493         body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
494         leading=12.5,
495         alignment=TA_LEFT, textColor=black, spaceAfter=5)
496         cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
497         leading=10.5,
498         textColor=black)
499         cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",

```



```

fontSize=8.2,
489             leading=10.5, textColor=black)
490     head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
491             leading=10.5, textColor=black)
492
493     rows = report.sorted_rows()
494     story: List[Any] = [
495         Paragraph("Signal Ablation & Feature-Enablement Report", title),
496         Paragraph(
497             f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
498             f"contribution & n={report.num_videos} golden matches, IoU
{report.iou}, "
499             f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
500             "Honest measured results only.",
501             subtitle),
502     ]
503
504     full_set = ", ".join(report.full_set_signals)
505     story.append(Paragraph("Run provenance", h2))
506     story.append(Paragraph(
507         f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
508         f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
509         f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
"
510         "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
a "
511         "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep
only "
512         "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
agreeing.",
513         body))
514
515     story.append(Paragraph("Marginal contribution per signal", h2))
516     table_data: List[List[Any]] = [[
517         Paragraph("Signal", head), Paragraph("Group", head),
518         Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
519         Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
520         Paragraph("Over-seg", head), Paragraph("Verdict", head),
521     ]]
522     for r in rows:
523         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
524         sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
525         overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
526         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
527         df1_style = cell_b if r.verdict == EARNNS_KEEP else cell
528         table_data.append([
529             Paragraph(r.signal, cell), Paragraph(r.group, cell),
530             Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
531             Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
cell),
532             Paragraph(overseg, cell), Paragraph(verdict, cell),
533         ])
534     col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
2.6 * cm]
535     tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
536     last = len(table_data) - 1
537     tbl.setStyle(TableStyle([
538         ("BACKGROUND", (0, 0), (-1, 0), light),
539         ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
540         ("TOPPADDING", (0, 0), (-1, -1), 4),
541         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
542         ("LEFTPADDING", (0, 0), (-1, -1), 4),
543         ("RIGHTPADDING", (0, 0), (-1, -1), 4),
544         ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),

```

```

545         ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
546         ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
547         ("LINEBELOW", (0, last), (-1, last), 1.0, black),
548         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
549     ]))
550     story.append(tbl)
551     story.append(Spacer(1, 0.3 * cm))
552
553     story.append(Paragraph("Verdict key", h2))
554     story.append(Paragraph(
555         "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
clearing 0 "
556         "and the sign test agreeing (the signal earns its keep). "
557         "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
the CI "
558         "spans 0 (keep, watch as the corpus grows). "
559         "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement
only after a "
560         "trend across multiple runs, never on one wash at small n. "
561         "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
562         "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
563         "never auto-flagged dead weight on 'no measured lift'.",
564         body))
565
566     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
567                             topMargin=1.8 * cm, bottomMargin=1.8 * cm,
568                             title="Signal Ablation & Feature-Enablement Report",
569                             author="badminton-highlight-indexer")
570     doc.build(story)
571     logger.info("wrote ablation PDF -> %s", path)
572     return path
573
574
575     # -----
CLI
576
577
578     def main() -> None:
579         ap = argparse.ArgumentParser(
580             description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
581                 "over experiment.compare + eval_stats; no new scoring math).")
582         ap.add_argument("--features-dir", default="output",
583                         help="dir of *_golden_features.json (the persisted golden
substrate)")
584         ap.add_argument("--granularity", choices=("signal", "group"), default="group",
585                         help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
586         ap.add_argument("--iou", type=float, default=0.5)
587         ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
588         ap.add_argument("--litmus", action="store_true",
589                         help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
590                 "instead of the learned full-set ablation")
591         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
592         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
593         args = ap.parse_args()
594
595         videos = load_videos(args.features_dir)
596         if len(videos) < 2:
597             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} ")

```

```

598         f"(found {len(videos)})")
599
600     if args.litmus:
601         cfg = litmus_config()
602         cfg.iou, cfg.seed = args.iou, args.seed
603     else:
604         cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
605                             granularity=args.granularity, iou=args.iou, seed=args.seed)
606
607     report = run_ablation(videos, cfg)
608     print(format_table(report))
609
610     if args.json:
611         with open(args.json, "w", encoding="utf-8") as f:
612             json.dump(report.as_dict(), f, indent=2)
613             print(f"\nwrote report JSON -> {args.json}")
614     if args.pdf:
615         export_pdf(report, args.pdf)
616         print(f"wrote PDF report -> {args.pdf}")
617
618
619 if __name__ == "__main__":
620     main()
621

```

9. user (2026-06-18T07:54:53.160Z)

```

1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here**; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations

```

```

39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
ratio
56 from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
scores
57
58 logger = logging.getLogger(__name__)
59
60 # Where a comparison run appends one reproducible record. Tracked location (NOT under
61 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62 # regression.DEFAULT_BASELINE_PATH.
63 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64 _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67 class ExperimentError(ValueError):
68     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69     score an unlabeled video with no pinned model, or a gate referencing an absent
70     feature)."""
71
72
73 # ----- Variant
74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the control behaviour (no gate, no learned
82     filter), so a variant that merely carries a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91       ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95       whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100     - ``feature_names`` ? the persisted features the learned filter consumes. When set

```

```

101         WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102         (honest) ? the recipe, not a pinned artifact.
103     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104       overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105     """
106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0
112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160

```

```

161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
181     """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
182     read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
183     ``fusion_audio.augment`` and older JSONs predate it).
184
185     The velocity-windowing / distill corpus-growth path persists a LEANER
186     ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts)
for clips
187     that never went through fusion feature extraction. The loaders that build cue-rich
188     ``VideoCandidates`` from these files ? ``ablation.load_videos``,
    ``fusion_compare.load_videos``
189     and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
    ``c["shuttle"]``,
190     so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
191     An empty candidate list is not a fusion file either.
192
193     Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
194     each landed an independent copy)."""
195     return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
196
197
198 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
199     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
200     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
201     col = vc.features.get(name)
202     n = len(vc.intervals)
203     if col is None:
204         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
205             name, vc.name)
206         return [0.0] * n
207     if len(col) != n:
208         raise ExperimentError(
209             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
210     return [float(x) for x in col]
211
212
213 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
214     cols = [_feature_column(vc, name) for name in feature_names]
215     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
216
217

```

```

218     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
219         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
220         n = len(vc.intervals)
221         mask = [True] * n
222         if variant.min_crossings > 0:
223             col = _feature_column(vc, "net_crossings")
224             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
225         if variant.min_players > 0:
226             col = _feature_column(vc, "players_in_court")
227             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
228         return mask
229
230
231     def predict(variant: Variant, vc: VideoCandidates,
232                 model: Optional[LogisticRegression] = None,
233                 threshold: Optional[float] = None,
234                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
235         """Variant prediction: gate by persisted features, optionally filter by a learned
236         model, then merge. Pure and deterministic.
237
238         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
239         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
240         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
241         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
242         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
243         them.
244
245         """
246         mask = _gate_mask(variant, vc)
247         if variant.feature_names:
248             if model is None:
249                 if variant.classifier_model is None:
250                     raise ExperimentError(
251                         f"variant {variant.name!r} is learned but has no model to predict "
252                         f"with (pass a fitted model or set classifier_model)")
253                 model = LogisticRegression.load(variant.classifier_model)
254             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
255             if calibrator is not None:
256                 proba = [calibrator(p) for p in proba]
257             thr = variant.threshold if threshold is None else threshold
258             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
259             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
260             return merge_overlaps(kept, gap_tol=variant.merge_gap)
261
262     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
263                             train_videos: Sequence[VideoCandidates], iou: float
264                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
265         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
266         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
267         (use the variant default). Honest: never looks at the held-out video.
268
269         """
270         calibrator: Optional[Callable[[float], float]] = None
271         if variant.calibrate:
272             raw: List[float] = []
273             y: List[int] = []
274             for vc in train_videos:
275                 raw.extend(float(p) for p in
276                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
277                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
278             if variant.calibrate == "platt":
279                 calibrator = fit_platt(raw, y)

```

```

278         elif variant.calibrate == "isotonic":
279             calibrator = fit_isotonic(raw, y)
280         else:
281             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
282         threshold: Optional[float] = None
283         if variant.tune_threshold:
284             best_t, best_f1 = variant.threshold, -1.0
285             for k in range(1, 20): # grid 0.05..0.95 on the train
fold's rally F1
286                 t = round(0.05 * k, 2)
287                 f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
288                             vc.gts or [], iou).f1 for vc in train_videos]
289                 mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
290                 if mean_f1 > best_f1:
291                     best_f1, best_t = mean_f1, t
292             threshold = best_t
293         return calibrator, threshold
294
295
296     # ----- variant scoring
297
298
299     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
300         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
301         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
302         X: List[List[float]] = []
303         y: List[int] = []
304         for vc in videos:
305             if vc.gts is None:
306                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
307             X.extend(_feature_matrix(vc, variant.feature_names or []))
308             y.extend(label_candidates(vc.intervals, vc.gts, iou))
309         if not X:
310             raise ExperimentError("cannot train a learned variant on zero candidates")
311         return LogisticRegression().fit(X, y, variant.feature_names)
312
313
314     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
315                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
316         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
317         mean aggregate.
318
319         Prediction policy:
320         - **static variant** (no learned filter): predict each video directly ? no
training,
321         so no leakage; per-video scoring is already honest.
322         - **learned, pinned model** (`classifier_model` set): score every video with the
SHIPPED model. ? optimistic if those videos were in its training set ? use this
323         to
324         report "how the shipped artifact does here", not for the promotion decision.
325         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
326         train on the OTHER videos, predict the held-out one. The number to trust.
327         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
328         """
329         for vc in videos:
330             if vc.gts is None:
331                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
332

```



```

333     per_video: List[Dict[str, Any]] = []
334     predictions: Dict[str, List[Interval]] = {}
335
336     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
337     pinned_model = (LogisticRegression.load(variant.classifier_model)
338                     if variant.classifier_model is not None else None)
339     all_model = None
340     if recipe_lovo and not honest:
341         all_model = _train(variant, videos, iou)
342
343     for i, vc in enumerate(videos):
344         cal: Optional[Callable[[float], float]] = None
345         thr: Optional[float] = None
346         if recipe_lovo and honest:
347             others = [v for j, v in enumerate(videos) if j != i]
348             if not others:
349                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
350             model: Optional[LogisticRegression] = _train(variant, others, iou)
351             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
352                 assert model is not None # _train never returns
None
353                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
354             elif recipe_lovo: # honest=False ? one model over all
videos
355                 model = all_model
356             else: # static variant or pinned model
357                 model = pinned_model
358             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
359             assert vc.gts is not None # guarded at function top
360             sc = score(preds, vc.gts, iou)
361             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
362             predictions[vc.name] = preds
363
364         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
365                     for m in _METRICS}
366     return {
367         "variant": variant.name,
368         "spec_hash": variant.spec_hash(),
369         "is_learned": variant.is_learned(),
370         "honest_lovo": recipe_lovo and honest,
371         "per_video": per_video,
372         "aggregate": aggregate,
373         "predictions": predictions,
374     }
375
376
377     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
378         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
379         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
380
381
382     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
383         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
384         gts_by_name = {v.name: (v.gts or []) for v in videos}
385         overlaps = _segm.DEFAULT_OVERLAPS
386         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
387         ratios: List[float] = []

```

```

388     for name, preds in eval_v.get("predictions", {}).items():
389         gts = gts_by_name.get(name, [])
390         got = _segm.f1_at_overlaps(preds, gts, overlaps)
391         for ov in overlaps:
392             f1k[ov].append(got[ov])
393         ratios.append(_segm.segment_count_ratio(preds, gts))
394
395     def _mean(xs: List[float]) -> float:
396         return sum(xs) / len(xs) if xs else 0.0
397     out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
398     f1k.items()}
399     out["segment_count_ratio"] = _mean(ratios)
400     return out
401
402     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
403                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
404         """C1 + C4 honest reporting layered over a compare() pair (additive,
405         EXPERIMENT_HARNESS §6).
406
407         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
408         order),
409         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
410         when
411         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
412         """
413         a_f1 = [p["f1"] for p in eval_a["per_video"]]
414         b_f1 = [p["f1"] for p in eval_b["per_video"]]
415         return {
416             "variant_a_ci": _ci_block(eval_a),
417             "variant_b_ci": _ci_block(eval_b),
418             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
419             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
420                             "variant_b": _segmentation_block(videos, eval_b)},
421         }
422
423     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
424               iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
425     Dict[str, Any]:
426         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
427         same
428         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
429
430         The deltas use the existing `metrics.score` (per video) +
431         `calibration.pct_improvement`;
432         learned variants are scored LOVO-honestly. The result is a self-contained record fit
433         for
434         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
435         it.
436
437         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
438         CIs, a
439         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
440         *alongside* the
441         existing bare-mean fields (additive: nothing existing changes). Set False for the
442         legacy
443         shape. This is the measurement-honesty wiring (ANALYZERS_AND_RECONCILERS.md
444         §13/C1+C4): a
445         bare LOVO mean at n?6 is not trustworthy on its own.
446         """
447         if len(videos) < 1:
448             raise ExperimentError("compare needs at least one labeled video")
449         eval_a = evaluate_variant(videos, variant_a, iou, honest)
450         eval_b = evaluate_variant(videos, variant_b, iou, honest)

```

```

440
441     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
442     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
443
444     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
445     per_video_delta = [
446         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
447         for p in eval_b["per_video"]
448     ]
449
450     # One fingerprint over the whole label set ? detects "the deltas were measured
against
451     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
452     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
453
454     result: Dict[str, Any] = {
455         "iou": iou,
456         "label_set_hash": gt_hash(all_gts),
457         "num_videos": len(videos),
458         "variant_a": eval_a,
459         "variant_b": eval_b,
460         "delta": delta,
461         "delta_pct": delta_pct,
462         "per_video_delta": per_video_delta,
463     }
464     if honest_stats:
465         result["honest"] = honest_summary(videos, eval_a, eval_b)
466     return result
467
468
469     # ----- SxS on unlabeled video
470
471
472     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
473         agree_iou: float = 0.5) -> Dict[str, Any]:
474         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
475
476         With no labels there is no lift to measure ? instead this surfaces where the two
477         variants **agree vs diverge**, which is exactly what a human reviews (and what two
478         highlight reels would show side by side):
479
480         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
481         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
482         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
483             the human / a later golden label adjudicates).
484
485         Both variants must be predictable without labels: a learned variant therefore needs
a
486         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
487         ``metrics.match_intervals`` ? no new matching logic.
488         """
489         preds_a = predict(variant_a, video)
490         preds_b = predict(variant_b, video)
491         mr = match_intervals(preds_a, preds_b, agree_iou)
492
493         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
494             for m in mr.matches]
495         agree_ious = [m.iou for m in mr.matches]
496         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
497         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
498
499         def _dur(ivs: List[Interval]) -> float:
500             return sum(e - s for s, e in ivs)

```

```

501
502     return {
503         "video": video.name,
504         "agree_iou": agree_iou,
505         "variant_a": variant_a.name,
506         "variant_b": variant_b.name,
507         "num_a": len(preds_a),
508         "num_b": len(preds_b),
509         "num_agreed": len(agreed),
510         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
511         "duration_a": _dur(preds_a),
512         "duration_b": _dur(preds_b),
513         "agreed": agreed,
514         "a_only": a_only,
515         "b_only": b_only,
516     }
517
518
519 # ----- leaderboard / DB
520
521
522 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
523                     timestamp: Optional[str] = None) -> Dict[str, Any]:
524     """Append a compact, reproducible record of a `compare()` result to the JSON
525     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
526     `regression_baseline.json`; deliberately the size of a baseline file, not a service
527     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
528     """
529     row: Dict[str, Any] = {
530         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
531         "iou": result.get("iou"),
532         "label_set_hash": result.get("label_set_hash"),
533         "num_videos": result.get("num_videos"),
534         "variant_a": {"name": result["variant_a"]["variant"],
535                     "spec_hash": result["variant_a"]["spec_hash"],
536                     "aggregate": result["variant_a"]["aggregate"]},
537         "variant_b": {"name": result["variant_b"]["variant"],
538                     "spec_hash": result["variant_b"]["spec_hash"],
539                     "aggregate": result["variant_b"]["aggregate"]},
540         "delta": result.get("delta"),
541     }
542     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
543     the
544     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
545     honest = result.get("honest") or {}
546     if honest.get("paired_delta_f1"):
547         row["honest_delta_f1"] = honest["paired_delta_f1"]
548     entries: List[Dict[str, Any]] = []
549     if os.path.isfile(path):
550         with open(path, encoding="utf-8") as f:
551             entries = json.load(f)
552     entries.append(row)
553     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
554     with open(path, "w", encoding="utf-8") as f:
555         json.dump(entries, f, indent=2)
556     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
557               row["variant_a"]["name"], row["variant_b"]["name"],
558               (result.get("delta") or {}).get("f1", 0.0), path)
559     return row
560
561 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
562                         gts: Optional[List[Interval]] = None,
563                         feature_names: Sequence[str] = ()) -> VideoCandidates:
564     """Bridge persisted detection ? `VideoCandidates` via

```

```

`Database.query_candidate_segments`.
565
566     ``feature_names`` selects which persisted features to lift out of each row's
567     ``stats``
568     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
569     function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
570     but works against any `Database` (including an in-memory one in tests).
571     """
572     rows = db.query_candidate_segments(video_id, detector=detector)
573     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
574     features: Dict[str, List[float]] = {}
575     for name in feature_names:
576         if name == "duration":
577             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
578         else:
579             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
580     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)

```

10. assistant (2026-06-18T07:54:56.849Z)

Now let me read the fourth file and check the test files.

11. user (2026-06-18T07:54:57.874Z)

```

1     """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2     over-segmentation-sensitive metric set, honestly.
3
4     This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` $4.5 /
$7)
5     measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6     IoU-F1 is blind to merged mega-windows, so we report it alongside segmental F1@k,
the
7     segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).
8
9     For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10    windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11    operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12    the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13    diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15    GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16    trajectory path + fps + frame_width; the GT rally intervals come from
17    ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19    Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20    """
21    from __future__ import annotations
22
23    import argparse

```

```

24     import json
25     import os
26     from typing import Any, Dict, List, Optional
27
28     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29     from backend.eval.metrics import Interval
30     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31
32     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
33     DEFAULT_FEATURES_DIR = "output"
34     DEFAULT_IOU = 0.5
35
36
37     # ----- #
38     # Pure scoring core (no I/O ? unit-testable)
39     # ----- #
40     def score_intervals(preds: List[Interval], gts: List[Interval],
41                       iou: float = DEFAULT_IOU,
42                       tau_start: Optional[float] = None,
43                       tau_end: Optional[float] = None) -> Dict[str, Any]:
44         """The full per-video metric row for predicted vs ground-truth rally intervals.
45
46         Pairs temporal-IoU P/R/F1 + boundary error (`metrics.score`) with the
47         over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
48         merged mega-window is *visible* even when IoU-F1 isn't moved.
49
50         Also carries the **R2** task-faithful block (`tolerance_metrics`, the headline
51         going forward;
52         tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
53         strict-1:1
54         tolerance-match `tol_f1/tol_precision/tol_recall` (over-production costs
55         precision) + the
56         start/end boundary-error medians + the over-seg guard counts. See
57         docs/R2_EVAL_METRIC_DESIGN.md.
58         """
59         from backend.eval import tolerance_metrics as tm
60         ts = tm.TAU_START if tau_start is None else tau_start
61         te = tm.TAU_END if tau_end is None else tau_end
62         sc = metrics.score(preds, gts, iou_threshold=iou)
63         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
64         ms = segmentation_metrics.merge_split_report(preds, gts)
65         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
66         be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
67         R4-aiming diagnostic)
68         osr = tm.over_seg_report(preds, gts)
69         return {
70             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
71             sc.mean_iou,
72             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
73             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
74             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
75             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
76             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
77             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
78             # --- R2 (tolerance-match) ? the task-faithful headline block ---
79             "tau_start": ts, "tau_end": te,
80             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
81             "tol_matched": float(len(tol_matches)),
82             "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
83             be["dend"]["abs_median"],
84             "split_count": osr["split_count"], "merge_count": osr["merge_count"],
85         }

```

```

81     _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
82                 "precision", "recall",
83                 "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
84
85
86     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
87         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
88
89         Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
90         leave-one-video-out discipline (windows within a video are correlated)."""
91         out: Dict[str, Any] = {"n": len(rows)}
92         for k in _AGG_KEYS:
93             vals = [r[k] for r in rows]
94             out[k] = eval_stats.mean_with_ci(vals) if vals else None
95         return out
96
97
98     # ----- #
99     # Golden corpus I/O
100    # ----- #
101    def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102                   features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103        """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
104        intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
105        with open(manifest_path) as fh:
106            manifest = json.load(fh)
107        out: List[Dict[str, Any]] = []
108        for v in manifest:
109            feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
110            gts: List[Interval] = []
111            if os.path.exists(feat):
112                with open(feat) as fh:
113                    gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
114            out.append({"name": v["name"], "trajectory": v["trajectory"],
115                       "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
"gts": gts})
116        return out
117
118
119    def windows_for(video: Dict[str, Any], preset: WindowingPreset,
120                   min_crossings: int = 0) -> List[Interval]:
121        """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
122
123        ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
124        ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
125        times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
126        from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
127        windowing (W1) leaves behind.
128
129        NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
130        drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
131        strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no

```

```

harm".
132     """
133     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
134     points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
135     wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
136     ivs = windowing.windows_to_intervals(wins)
137     if min_crossings > 0 and ivs:
138         from backend.pipeline.detectors.rally_gate import gate_windows
139         gated = [(g.start, g.end)
140                  for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
141                  if g.kept]
142         # Gating emptied a non-empty video ? keep the ungated windows.
143         ivs = gated if gated else ivs
144     return ivs
145
146
147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                 iou: float = DEFAULT_IΟΥ, min_crossings: int = 0,
149                 tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)
157             row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161                "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IΟΥ, base_min_crossings: int = 0,
166                 cand_min_crossings: int = 0,
167                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168                 guard_weight_merge: Optional[float] = None,
169                 guard_weight_split: Optional[float] = None,
170                 guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
172         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
175         alongside so the refinement is auditable."""
176         from backend.eval import tolerance_metrics as tm
177         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
178         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
179         by_name_b = {r["name"]: r for r in b["rows"]}
180         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
181         delta: Dict[str, Any] = {}
182         for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
183             delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
184                                                    [rc[k] for _, rc in paired])
185         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
186         if guard_weight_merge is not None:

```



```

187         gkw["weight_merge"] = guard_weight_merge
188     if guard_weight_split is not None:
189         gkw["weight_split"] = guard_weight_split
190     guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
191     return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
192
193
194     # ----- #
195     # Reporting
196     # ----- #
197     def _ci(m: Optional[Dict[str, Any]]) -> str:
198         return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"
199
200
201     def _format_guard(g: Dict[str, Any]) -> str:
202         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
203         basis = "rate" if g["use_rates"] else "count"
204         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
205                 f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
206                 f" NET verdict: {'PASS' if g['passes'] else 'BLOCK'} ",
207                 f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; ",
208                 f"no_merge_rate_regress={g['no_merge_rate_regress']})",
209                 f" strict per-video rule (R2 §2.3.3, for contrast): ",
210                 f"{'pass' if g['strict_passes'] else 'BLOCK'} ",
211                 f"(count increases ? merges:{len(g['new_merges'])})
splits:{len(g['new_splits'])})"]
212         if g["merge_rate_regress"]:
213             names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
214                               for r in g["merge_rate_regress"])
215             lines.append(f" ? merge_rate REGRESSED on: {names}")
216         return "\n".join(lines)
217
218
219     def format_report(result: Dict[str, Any]) -> str:
220         p = result["preset"]
221         bounded = []
222         if p.get("max_window_duration"):
223             bounded.append(f"cap={p['max_window_duration']} + ("hard)" if
p.get("hard_cap") else "")
224             if p.get("inactivity_end"):
225                 bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}
f"/md={p['inactivity_min_density']}")
226         if p.get("blob_fallback"):
227             bounded.append("blob_fallback")
228         extra = (" " + ", ".join(bounded)) if bounded else ""
229         lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
230                 f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
231                 f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}", ""]
232         lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
233         lines.append(" |---|---|---|---|---|---|---|")
234         for r in result["rows"]:
235             lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
236                          f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
237                          f"{r['num_pred']}/{r['num_gt']} |")
238         a = result["aggregate"]
239         lines += [f"***Aggregate (n={a['n']}, mean [95% CI]):** "
240                  f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
_ci(a['f1@0.5'])} · "

```

```

242         f"merge_rate {_ci(a['merge_rate'])} . seg_ratio
{_ci(a['segment_count_ratio'])}"]
243     # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
244     rows = result["rows"]
245     if rows and "tol_f1" in rows[0]:
246         ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
247         lines += [f"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
1:1)", "",
248                 "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
249                 "|---|---|---|---|---|---|---|---|"]
250     for r in rows:
251         lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
252                     f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
253                     f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} | "
{int(r['merge_count'])} | ")
254         lines += [f"***R2 aggregate (n={a['n']})** tolF1 {_ci(a['tol_f1'])} . "
255                 f"tolP {_ci(a['tol_precision'])} . tolR {_ci(a['tol_recall'])} . "
256                 f"|?start| {_ci(a['dstart_abs_median'])} . |?end|
{_ci(a['dend_abs_median'])} "
257                 f"(start?solved / end=the gap)"]
258     return "\n".join(lines)
259
260
261     def _resolve_preset(name: str) -> WindowingPreset:
262         if name in PRESETS:
263             return PRESETS[name]
264         raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")
265
266
267     def main() -> None:
268         ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
269         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
270         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
271         ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
272         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
273         ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
274         ap.add_argument("--min-crossings", type=int, default=0,
275                         help="net-crossings rally-state gate on --preset (0=off; W2
Gate-A)")
276         ap.add_argument("--vs-min-crossings", type=int, default=0,
277                         help="net-crossings gate on --vs (default: same as --min-
crossings)")
278         ap.add_argument("--max-window-duration", type=float, default=None,
279                         help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
12)")
280         ap.add_argument("--vs-max-window-duration", type=float, default=None,
281                         help="cap (s) on --vs ONLY (default: same as --max-window-duration);
lets a "
282                         "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
one shot")
283         ap.add_argument("--hard-cap", action="store_true",
284                         help="enforce the cap as a HARD cap on --preset (chop continuously-
active "
285                         "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
286         ap.add_argument("--vs-hard-cap", action="store_true",
287                         help="hard-cap fallback on --vs (default: same as --hard-cap)")
288         ap.add_argument("--blob-fallback", action="store_true",
289                         help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
290                         "cap at its midpoint + flag/log it; the continuously-active

```

```

blob last resort)")
291     ap.add_argument("--vs-blob-fallback", action="store_true",
292                     help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
293     ap.add_argument("--blob-factor", type=float, default=None,
294                     help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
295     ap.add_argument("--inactivity-end", type=float, default=None,
296                     help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
297     ap.add_argument("--vs-inactivity-end", type=float, default=None,
298                     help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
299     ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
300                     help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
301     ap.add_argument("--inactivity-min-density", type=float, default=None,
302                     help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
303     ap.add_argument("--tau-start", type=float, default=None,
304                     help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
305     ap.add_argument("--tau-end", type=float, default=None,
306                     help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
307     ap.add_argument("--guard-weight-merge", type=float, default=None,
308                     help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
309     ap.add_argument("--guard-weight-split", type=float, default=None,
310                     help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
311     ap.add_argument("--guard-counts", action="store_true",
312                     help="R1 over-seg guard scores raw counts instead of normalized
rates "
313                             "(rates are the default ? comparable across a structural
mega?bounded change)")
314     ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
315     args = ap.parse_args()
316
317     from dataclasses import replace
318     golden = load_golden(args.manifest, args.features_dir)
319     def _apply_inactivity(preset, end_val):
320         if end_val is not None:
321             preset = replace(preset, inactivity_end=end_val)
322         if args.inactivity_rally_thresh is not None:
323             preset = replace(preset,
inactivity_rally_thresh=args.inactivity_rally_thresh)
324         if args.inactivity_min_density is not None:
325             preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
326         return preset
327
328     base = _resolve_preset(args.preset)
329     if args.max_window_duration is not None:
330         base = replace(base, max_window_duration=args.max_window_duration)
331     if args.hard_cap:
332         base = replace(base, hard_cap=True)
333     if args.blob_fallback:
334         base = replace(base, blob_fallback=True)
335     if args.blob_factor is not None:
336         base = replace(base, blob_factor=args.blob_factor)
337     base = _apply_inactivity(base, args.inactivity_end)
338     if args.vs:
339         vs_mc = args.vs_min_crossings or args.min_crossings
340         cand = _resolve_preset(args.vs)
341         cand_cap = (args.vs_max_window_duration if args.vs_max_window_duration is not

```

```

None
342         else args.max_window_duration)
343     if cand_cap is not None:
344         cand = replace(cand, max_window_duration=cand_cap)
345     if args.vs_hard_cap or args.hard_cap:
346         cand = replace(cand, hard_cap=True)
347     if args.vs_blob_fallback or args.blob_fallback:
348         cand = replace(cand, blob_fallback=True)
349     if args.blob_factor is not None:
350         cand = replace(cand, blob_factor=args.blob_factor)
351     cand = _apply_inactivity(cand, args.vs_inactivity_end if args.vs_inactivity_end
is not None
352         else args.inactivity_end)
353     result = compare(golden, base, cand, args.iou,
354                     base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
355                     tau_start=args.tau_start, tau_end=args.tau_end,
356                     guard_weight_merge=args.guard_weight_merge,
357                     guard_weight_split=args.guard_weight_split,
358                     guard_use_rates=not args.guard_counts)
359     print(format_report(result["base"]))
360     print("\n" + format_report(result["cand"]))
361     d = result["delta"]
362     print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
363           f"n_paired={result['n_paired']}")
364     for k, dd in d.items():
365         st = dd["sign_test"]
366         print(f" ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f},{dd['ci_high']:+.3f}] "
367               f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
368     print(_format_guard(result["over_seg_guard"]))
369     else:
370         result = evaluate(golden, base, args.iou, args.min_crossings,
371                           tau_start=args.tau_start, tau_end=args.tau_end)
372         print(format_report(result))
373     if args.json_out:
374         with open(args.json_out, "w", encoding="utf-8") as fh:
375             json.dump(result, fh, indent=2)
376         print(f"\n[written {args.json_out}]")
377
378
379 if __name__ == "__main__":
380     main()
381

```

12. assistant (2026-06-18T07:55:00.675Z)

Now let me look at the test files to assess coverage honestly. Let me read the key test files for these four modules.

13. user (2026-06-18T07:55:01.187Z)

```

1     """Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic
Tier-0.
2
3     Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate
is
4     **parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the
replay
5     path), and asserts the Tier-0 privacy invariant positively (no
person/audio/pose/gait/face keys).
6
7     S6 honest-limits caveat: green here proves only that the deterministic post-trajectory
transforms

```

```

8     are unchanged for the frozen synthetic cases ? characterization is behaviour-locking,
NOT
9     correctness, and synthetic-tier numbers measure plumbing, not field quality.
10    """
11    import json
12
13    import pytest
14
15    from backend.eval import golden_fixtures as gf
16
17    # Slugs discovered from the committed manifest ? the suite grows by accretion
automatically.
18    MANIFEST = gf.load_manifest()
19    SLUGS = [entry["slug"] for entry in MANIFEST]
20
21
22    def test_manifest_is_non_empty():
23        """A committed manifest with at least one fixture must exist (else nothing is
gated)."""
24        assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-
synthetic"
25
26
27    @pytest.mark.parametrize("slug", SLUGS)
28    def test_replay_matches_committed_expected(slug):
29        """Replay the fixture and parse-compare against its committed expected.json.
30
31        Characterization = behaviour-locking, NOT correctness (§6): a non-empty mismatch
list means
32        the deterministic post-trajectory behaviour CHANGED for this frozen case, not that
it is wrong.
33        """
34        committed = gf.load_fixture(slug)["expected"]
35        recomputed = gf.replay_fixture(slug)
36        mismatches = gf.compare_expected(recomputed, committed)
37        assert mismatches == [], (
38            f"{slug}: behaviour changed vs frozen snapshot (characterization, not
correctness ? §6):\n"
39            + "\n".join(f" - {m}" for m in mismatches)
40        )
41
42
43    @pytest.mark.parametrize("slug", SLUGS)
44    def test_schema_version_le_current(slug):
45        """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-
compatible)."""
46        fx = gf.load_fixture(slug)
47        assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
48        assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA
49
50
51    @pytest.mark.parametrize("slug", SLUGS)
52    def test_no_person_or_audio_keys(slug):
53        """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key
appears
54        anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
55        fdir = gf.FIXTURES_DIR / slug
56        raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
57        for forbidden in gf.FORBIDDEN_KEYS:
58            assert forbidden not in raw, (
59                f"{slug}: forbidden key substring {forbidden!r} present in expected.json
(Tier-0 leak)")
60
61        expected = gf.load_fixture(slug)["expected"]
62        for i, cand in enumerate(expected["candidates"]):

```

```

63         # The only feature block is the shuttle block, and it holds EXACTLY the 5
shuttle cols.
64         assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}]
has extra keys"
65         assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
66             f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")
67
68
69     @pytest.mark.parametrize("slug", SLUGS)
70     def test_tier0_provenance_tags_by_kind(slug):
71         """Tier-0 hard tags, asserted BY KIND so the suite stays green when a
`cleared_real` fixture
72         is added (red-team #11: the old test hard-asserted kind=='synthetic' for ALL slugs,
which a
73         future real fixture would red). Every Tier-0 fixture must carry a license +
minors_free=True;
74         a synthetic one is additionally license=='synthetic', a cleared_real one is
owned==True."""
75         prov = gf.load_fixture(slug)["provenance"]
76         kind = prov["kind"]
77         assert kind in ("synthetic", "cleared_real"), f"unexpected Tier-0 kind {kind!r}"
78         # Common to ALL Tier-0 fixtures: non-blank license + minors excluded.
79         assert isinstance(prov.get("license"), str) and prov["license"].strip(),
"blank/missing license"
80         assert prov["minors_free"] is True
81         if kind == "synthetic":
82             assert prov["license"] == "synthetic"
83         else: # cleared_real
84             assert prov["owned"] is True
85         # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
86         assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}
87
88
89     def test_control_is_not_learned():
90         """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression
? D5)."""
91         from backend.eval.experiment import CONTROL
92         assert CONTROL.is_learned() is False
93
94
95     def test_perturbing_trajectory_changes_segments(tmp_path):
96         """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving
the
97         windows are RE-DERIVED from the CSV each run, not merely echoed from
expected.json."""
98         slug = SLUGS[0]
99         src = gf.FIXTURES_DIR / slug
100         work = tmp_path / slug
101         work.mkdir()
102         # Copy the fixture, then TRUNCATE the trajectory to its first half so the derived
window's
103         # `end` shrinks ? a guaranteed structural change in the re-derived segments.
(Deleting an
104         # interior hole would not move start/end, since merge_gap coalesces across it and
the frame
105         # indices of the surviving rows are unchanged.)
106         lines = (src / "trajectory.csv").read_text(encoding="utf-8").splitlines()
107         header, body = lines[0], lines[1:]
108         mutated = [header] + body[: len(body) // 2]
109         (work / "trajectory.csv").write_text("\n".join(mutated) + "\n", encoding="utf-8",
newline="\n")
110         (work / "labels.csv").write_text((src / "labels.csv").read_text(encoding="utf-8"),
encoding="utf-8", newline="\n")
111         (work / "provenance.json").write_text((src /
"provenance.json").read_text(encoding="utf-8"),

```

```

113             encoding="utf-8", newline="\n")
114
115     original = gf.load_fixture(slug)["expected"]
116     perturbed = gf.replay_fixture(work)
117     assert perturbed["control_segments"] != original["control_segments"], (
118         "perturbed trajectory produced identical segments ? replay is echoing, not re-
deriving")
119
120
121     def test_freeze_is_idempotent(tmp_path):
122         """Re-freezing every built-in fixture into a temp dir produces expected.json
structurally
123         equal to the committed one (correct-by-construction + deterministic)."""
124         slugs = gf.freeze_synthetic(fixtures_dir=tmp_path)
125         assert slugs, "freeze_synthetic produced no fixtures"
126         for slug in slugs:
127             committed = gf.load_fixture(slug)["expected"]
128             refrozen = json.loads((tmp_path / slug /
"expected.json").read_text(encoding="utf-8"))
129             # Structural (parsed) equality ? no float churn, exact same shape/values.
130             assert gf.compare_expected(refrozen, committed) == [], (
131                 f"{slug}: re-frozen expected.json differs from committed (non-idempotent
freeze)")
132             assert refrozen == committed, f"{slug}: re-frozen expected.json not structurally
identical"
133
134
135     def test_check_cli_passes(capsys):
136         """The --check CLI replays all fixtures and exits 0 when they match the committed
snapshots."""
137         rc = gf.main(["--check"])
138         assert rc == 0
139

```

14. user (2026-06-18T07:55:04.628Z)

```

1     """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3     These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4     minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5     its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7     Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8     We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
and the
9     ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10    prove:
11        * the refusal gate blocks every unsafe combination (M2);
12        * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13        * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14        ``control_segments`` shift toward 0);
15        * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17    $6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18    are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.
19    """

```

```

20     import pytest
21
22     from backend.eval import golden_fixtures as gf
23
24
25     # -----
26     # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27     # -----
28
29
30     def _write_source(tmp_path, *, frame_offset: int = 0):
31         """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33         Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34         ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35         capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36         """
37         spec = gf.builtin_specs()[0] # clean_single_rally
38         fps, fw = spec.fps, spec.frame_width
39         rows = gf.make_synthetic_trajectory(spec)
40         if frame_offset:
41             rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
42         gts = list(spec.gts)
43         if frame_offset:
44             off_sec = frame_offset / fps
45             gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
46
47         traj = tmp_path / "src_trajectory.csv"
48         labels = tmp_path / "src_labels.csv"
49         gf.write_trajectory_csv(rows, traj)
50         gf.write_labels_csv(gts, labels)
51         return traj, labels, fps, fw, gts
52
53
54     # -----
55     # M2 ? refusal gate
56     # -----
57
58
59     @pytest.mark.parametrize(
60         "label, kwargs, why",
61         [
62             ("scenario", {"minors_free": False, "owned": True, "license": "owned"},
"minors_free=False"),
63             ("scenario", {"minors_free": True, "owned": False, "license": "owned"},
"owned=False"),
64             ("scenario", {"minors_free": True, "owned": True, "license": ""}, "license=''),
65             ("scenario", {"minors_free": True, "owned": True, "license": "   "},
"license=blank"),
66             ("scenario", {"minors_free": True, "owned": True, "license": None},
"license=None"),
67             # red-team #1: a free-text identifier label (name / event / spaces /
punctuation) is refused.
68             ("Lin Dan vs Lee, India Open 2024",
69              {"minors_free": True, "owned": True, "license": "owned"}, "label=player-name
free text"),
70             ("mens singles final",
71              {"minors_free": True, "owned": True, "license": "owned"}, "label=spaces"),
72             ("Final_2024", {"minors_free": True, "owned": True, "license": "owned"},
"label=uppercase"),
73             ("", {"minors_free": True, "owned": True, "license": "owned"}, "label=empty"),
74         ],
75     )
76     def test_refusal_gate_blocks_unsafe(tmp_path, label, kwargs, why):

```



```

77         """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
blank license
78         AND a STRICT-slug label (no free-text identifier ? red-team #1).
79
80         The refusal is the code-enforced provenance gate (§4c / §7 ?): the unsafe public-
fixture path
81         must be impossible to reach silently. A refused freeze leaves NO partial fixture dir
(red-team
82         #3b: write-to-temp-then-promote).
83         """
84         traj, labels, fps, fw, _ = _write_source(tmp_path)
85         with pytest.raises(ValueError) as ei:
86             gf.freeze_real_fixture(
87                 label, traj, labels,
88                 fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)
89         # RefusalError is a ValueError subclass; the message must name a reason.
90         assert isinstance(ei.value, gf.RefusalError), why
91         assert "REFUSED" in str(ei.value), why
92         # Nothing was written for a refused freeze (no fixture dir, partial or otherwise).
93         assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
"fixtures").iterdir()), (
94             f"{why}: a refused freeze must not write any fixture")
95
96
97         # -----
98         # P1 ? de-attribution
99         # -----
100
101
102     def test_real_fixture_is_deattributed(tmp_path):
103         """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
104         no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
105         fixtures = tmp_path / "fixtures"
106         traj, labels, fps, fw, _ = _write_source(tmp_path)
107         expected = gf.freeze_real_fixture(
108             "clean_rally", traj, labels,
109             license="owned", minors_free=True, owned=True,
110             fps=fps, frame_width=fw, fixtures_dir=fixtures)
111
112         slug = expected["slug"]
113         assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"
114
115         prov = gf._read_json(fixtures / slug / "provenance.json")
116         assert prov["kind"] == "cleared_real"
117         assert prov["minors_free"] is True and prov["owned"] is True
118         assert prov["license"] == "owned"
119         assert prov["slug"] == slug
120         assert prov["label"] == "clean_rally" # strict-slug label is committed verbatim
121         # No attribution keys / no source filename anywhere in the provenance. `source_note`
is now
122         # DROPPED entirely (red-team #1b): operator free-text never reaches the committed
bytes.
123         for banned in ("video_id", "name", "filename", "path", "match", "capture_date",
"source_note"):
124             assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
125             prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
126             for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
127                 assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
128
129         # expected.json carries no biometric/identity/non-shuttle stream.
130         exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()

```

```

131         for forbidden in gf.FORBIDDEN_KEYS:
132             assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
133
134         # The fixture is registered in the manifest with kind=cleared_real.
135         manifest = gf.load_manifest(fixtures)
136         rows = [e for e in manifest if e["slug"] == slug]
137         assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
138
139
140     def test_supplied_slug_is_honored(tmp_path):
141         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
142         fixtures = tmp_path / "fixtures"
143         traj, labels, fps, fw, _ = _write_source(tmp_path)
144         expected = gf.freeze_real_fixture(
145             "scenario", traj, labels, slug="fx_custom00",
146             license="owned", minors_free=True, owned=True,
147             fps=fps, frame_width=fw, fixtures_dir=fixtures)
148         assert expected["slug"] == "fx_custom00"
149         assert (fixtures / "fx_custom00" / "expected.json").is_file()
150
151
152     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
153         """Freezing a second real fixture must APPEND to the manifest, not wipe the
first."""
154         fixtures = tmp_path / "fixtures"
155         traj, labels, fps, fw, _ = _write_source(tmp_path)
156         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
157                                     license="owned", minors_free=True, owned=True,
158                                     fps=fps, frame_width=fw, fixtures_dir=fixtures)
159         e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
160                                     license="owned", minors_free=True, owned=True,
161                                     fps=fps, frame_width=fw, fixtures_dir=fixtures)
162         slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
163         assert {e1["slug"], e2["slug"]} <= slugs
164
165
166     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
167         """source_note is ACCEPTED-BUT-NOT-PERSISTED (red-team #1b), yet still scanned
defensively ? a
168         path/name in it trips the positive scan and refuses (and leaves no partial dir, red-
team #3b)."""
169         fixtures = tmp_path / "fixtures"
170         traj, labels, fps, fw, _ = _write_source(tmp_path)
171         with pytest.raises(gf.RefusalError):
172             gf.freeze_real_fixture(
173                 "scenario", traj, labels, slug="fx_leak0001",
174                 license="owned", minors_free=True, owned=True,
175                 fps=fps, frame_width=fw, fixtures_dir=fixtures,
176                 source_note=str(traj)) # leaking the source path ? must be refused
177         # The refused freeze left no fixture dir behind (write-to-temp-then-promote).
178         assert not (fixtures / "fx_leak0001").exists(), "refused freeze must not leave a
partial dir"
179
180
181     def test_source_note_safe_is_not_persisted(tmp_path):
182         """A SAFE source_note (no path/name) is accepted but NEVER written to the committed
provenance
183         (red-team #1b: operator free-text is an attribution back-channel, dropped from
committed bytes)."""
184         fixtures = tmp_path / "fixtures"
185         traj, labels, fps, fw, _ = _write_source(tmp_path)
186         expected = gf.freeze_real_fixture(
187             "scenario", traj, labels, slug="fx_note0001",

```

```

188         license="owned", minors_free=True, owned=True,
189         fps=fps, frame_width=fw, fixtures_dir=fixtures,
190         source_note="benign generic note")
191     prov = gf._read_json(fixtures / expected["slug"] / "provenance.json")
192     assert "source_note" not in prov, "source_note must NOT be persisted into
provenance"
193     prov_text = (fixtures / expected["slug"] /
"provenance.json").read_text(encoding="utf-8")
194     assert "benign generic note" not in prov_text, "source_note text leaked into
committed bytes"
195
196
197     def test_freeze_real_label_slug_committed(tmp_path):
198         """A valid strict-slug label is committed verbatim; a free-text label is refused
(red-team #1a)."""
199         fixtures = tmp_path / "fixtures"
200         traj, labels, fps, fw, _ = _write_source(tmp_path)
201         ok = gf.freeze_real_fixture(
202             "mens-singles_final-01", traj, labels, slug="fx_lab00001",
203             license="owned", minors_free=True, owned=True,
204             fps=fps, frame_width=fw, fixtures_dir=fixtures)
205         prov = gf._read_json(fixtures / ok["slug"] / "provenance.json")
206         assert prov["label"] == "mens-singles_final-01"
207         with pytest.raises(gf.RefusalError):
208             gf.freeze_real_fixture(
209                 "Lin Dan vs Lee 2024", traj, labels, slug="fx_lab00002",
210                 license="owned", minors_free=True, owned=True,
211                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
212
213
214     def test_freeze_real_negative_reset_label_refused(tmp_path):
215         """A label timeline that predates the earliest trajectory frame ? negative reset
start ? refused
216         BEFORE any write, leaving no partial fixture dir (red-team #3a)."""
217         fixtures = tmp_path / "fixtures"
218         spec = gf.builtin_specs()[0]
219         fps, fw = spec.fps, spec.frame_width
220         # Trajectory starts at frame 300 (t0 ? 10s) but a label starts at t=0 ? reset start
< 0.
221         rows = [(f + 300, v, x, y) for (f, v, x, y) in gf.make_synthetic_trajectory(spec)]
222         traj = tmp_path / "src_trajectory.csv"
223         labels = tmp_path / "src_labels.csv"
224         gf.write_trajectory_csv(rows, traj)
225         gf.write_labels_csv([(0.0, 4.0)], labels)
226         with pytest.raises(gf.RefusalError) as ei:
227             gf.freeze_real_fixture(
228                 "scenario", traj, labels, slug="fx_neg00001",
229                 license="owned", minors_free=True, owned=True,
230                 fps=fps, frame_width=fw, fixtures_dir=fixtures, time_origin_reset=True)
231         assert "predates the trajectory" in str(ei.value)
232         assert not (fixtures / "fx_neg00001").exists(), "refused freeze must not leave a
partial dir"
233
234
235     def test_scan_forbidden_catches_label_csv_source_stem(tmp_path):
236         """red-team #4: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``)
must still
237         be caught ? the source-path scan uses EXACT schema-literal membership, never
substring-skip."""
238         # A source-path token "label" is NOT an exact schema literal, so it must trip the
scan when it
239         # appears in the written text. We feed it via a source path whose stem is "label".
240         text = '{"slug": "fx_x", "note": "label appears here"}'
241         with pytest.raises(gf.RefusalError) as ei:
242             gf._scan_forbidden(text, where="x/expected.json",

```

```

243         source_paths=["/some/dir/label.csv"])
244     assert "source-path token" in str(ei.value) and "label" in str(ei.value)
245
246
247     def test_scan_forbidden_allows_exact_schema_literal(tmp_path):
248         """A source path that EXACTLY equals a schema literal (e.g. ``trajectory.csv``) is
not a leak ?
249         those bytes are written regardless, so the scan must not false-positive on them."""
250         text = '{"slug": "fx_x", "paths": {"trajectory": "trajectory.csv", "labels":
"labels.csv"}}'
251         # Source basename == schema literal "trajectory.csv": must NOT raise.
252         gf._scan_forbidden(text, where="x/expected.json",
source_paths=["/some/dir/trajectory.csv"])
253
254
255         # -----
256         # P1 (d) ? time-origin reset is metric-invariant
257         # -----
258
259
260     def _raw_score_of_frozen(fdir):
261         """Re-score a frozen fixture from its committed CSVs through the SHIPPED stack
WITHOUT the
262         snapshot's final 6dp rounding ? so the test sees the genuine metric, not the
serialization grid.
263
264         (``replay_fixture`` rounds every float to 6dp for the committed snapshot; on a sub-
second
265         time-origin reset two boundary errors can land on adjacent 6dp grid points ? a 1e-6
SERIALIZED
266         gap that is the grid, not a metric divergence. The raw score below exposes the true
delta.)
267         """
268         from backend.eval import distill_local
269         from backend.eval.experiment import CONTROL, VideoCandidates, predict
270         from backend.eval.gt_loader import load_gt_intervals
271         from backend.eval.metrics import score
272         intervals, feature_rows = distill_local.build_candidates(
273             str(fdir / "trajectory.csv"), fps=30.0, frame_width=1280.0, proposal=(0.02, 2.0,
2.0))
274         feats = {n: [r[i] for r in feature_rows] for i, n in enumerate(gf.SHUTTLE_FEATURES)}
275         gts = load_gt_intervals(str(fdir / "labels.csv"), strict=True)
276         preds = predict(CONTROL, VideoCandidates("x", intervals, feats, gts))
277         return score(preds, gts, 0.5)
278
279
280     # Fractional-second offsets (7, 11, 13 frames @30fps) plus whole-second (9000 = 300.0s)
and a few
281     # more sub-second offsets ? every one must be metric-invariant within _FLOAT_TOL (red-
team #2).
282     @pytest.mark.parametrize("offset_frames", [1, 7, 11, 13, 17, 9000])
283     def test_time_origin_reset_is_metric_invariant(tmp_path, offset_frames):
284         """A source frozen WITH reset scores within ``_FLOAT_TOL`` of the SAME source frozen
WITHOUT
285         reset, for ANY fractional-second offset ? and its absolute control_segments shift
toward 0
286         (severed timeline). Before fix #2 the boundary error drifted to 1.000000000001e-06
(just past
287         _FLOAT_TOL) on a sub-second offset; rounding t0_sec + each label endpoint to 6dp
before the
288         shift keeps the genuine (raw) delta at the float-error floor (~3.3e-7)."""
289         traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
290
291         with_reset = gf.freeze_real_fixture(
292             "scenario", traj, labels, slug="fx_reset_on",

```

```

293         license="owned", minors_free=True, owned=True,
294         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
295         time_origin_reset=True)
296     without_reset = gf.freeze_real_fixture(
297         "scenario", traj, labels, slug="fx_reset_off",
298         license="owned", minors_free=True, owned=True,
299         fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
300         time_origin_reset=False)
301
302     # (1) Structural fields (counts/TP/FP/FN) are EXACTLY equal regardless of offset.
303     for k in ("num_pred", "num_gt", "true_positives", "false_positives",
304 "false_negatives"):
305         assert with_reset["score"][k] == without_reset["score"][k], f"{k} changed under
reset"
306
307     # (2) Every float score field is metric-invariant within _FLOAT_TOL on the RAW (un-
rounded)
308     #     score ? the genuine metric, not the 6dp serialization grid (see
_raw_score_of_frozen).
309     raw_on = _raw_score_of_frozen(tmp_path / "with" / "fx_reset_on")
310     raw_off = _raw_score_of_frozen(tmp_path / "without" / "fx_reset_off")
311     for f in ("precision", "recall", "f1", "mean_iou", "mean_start_error",
312 "mean_end_error"):
313         delta = abs(getattr(raw_on, f) - getattr(raw_off, f))
314         assert delta <= gf._FLOAT_TOL, (
315             f"offset={offset_frames}: {f} not metric-invariant under time-origin reset:
"
316             f"|?|={delta!r} > _FLOAT_TOL={gf._FLOAT_TOL!r}")
317
318     # (3) A whole-SECOND offset is additionally BIT-identical on the serialized snapshot
(the 6dp
319     #     grid lands cleanly), proving the reset itself introduces no real divergence.
320     if offset_frames % int(fps) == 0:
321         assert with_reset["score"] == without_reset["score"], (
322             "whole-second offset must be bit-identical on the serialized snapshot")
323
324     # (4) Absolute timeline differs ? the reset pulls control_segments toward 0.
325     seg_on = with_reset["control_segments"]
326     seg_off = without_reset["control_segments"]
327     assert len(seg_on) == len(seg_off) and len(seg_on) >= 1
328     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
329     shift = seg_off[0][0] - seg_on[0][0]
330     assert abs(shift - offset_frames / fps) < 1e-3, (
331         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
332
333     # Reset trajectory starts at frame 0.
334     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
335         .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
336     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
337
338     # -----
339     # P1 (e/f) ? round-trip / correct-by-construction
340     # -----
341
342     def test_freeze_real_round_trips_check(tmp_path):
343         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
344         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
345         fixtures = tmp_path / "fixtures"
346         traj, labels, fps, fw, _ = _write_source(tmp_path)
347         expected = gf.freeze_real_fixture(

```

```

347         "scenario", traj, labels, slug="fx_rt000001",
348         license="owned", minors_free=True, owned=True,
349         fps=fps, frame_width=fw, fixtures_dir=fixtures)
350
351     fdir = fixtures / expected["slug"]
352     committed = gf._read_json(fdir / "expected.json")
353     recomputed = gf.replay_fixture(fdir)
354     assert gf.compare_expected(recomputed, committed) == [], (
355         "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
356
357
358     def test_freeze_real_uses_strict_label_loader(tmp_path):
359         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
360         never silently shrinks the GT set."""
361         fixtures = tmp_path / "fixtures"
362         traj, labels, fps, fw, _ = _write_source(tmp_path)
363         # Corrupt one label row (end <= start) ? strict loader must raise.
364         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
365         with pytest.raises(ValueError):
366             gf.freeze_real_fixture(
367                 "scenario", traj, labels, slug="fx_strict01",
368                 license="owned", minors_free=True, owned=True,
369                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
370
371
372     def test_freeze_real_refuses_to_clobber_existing_dir(tmp_path):
373         """Freezing into a slug dir that already exists is refused (no silent overwrite)."""
374         fixtures = tmp_path / "fixtures"
375         traj, labels, fps, fw, _ = _write_source(tmp_path)
376         gf.freeze_real_fixture("a", traj, labels, slug="fx_dup00001",
377                                license="owned", minors_free=True, owned=True,
378                                fps=fps, frame_width=fw, fixtures_dir=fixtures)
379         with pytest.raises(ValueError):
380             gf.freeze_real_fixture("b", traj, labels, slug="fx_dup00001",
381                                    license="owned", minors_free=True, owned=True,
382                                    fps=fps, frame_width=fw, fixtures_dir=fixtures)
383
384
385     # -----
386     # FAILURE semantics ? the gates must FAIL loudly, not silently pass (red-team #10, #5,
#7).
387     # -----
388
389
390     def _good_snapshot(tmp_path):
391         """Freeze a clean real fixture and return its committed expected.json snapshot
dict."""
392         fixtures = tmp_path / "fixtures"
393         traj, labels, fps, fw, _ = _write_source(tmp_path)
394         return gf.freeze_real_fixture(
395             "scenario", traj, labels, slug="fx_snap00001",
396             license="owned", minors_free=True, owned=True,
397             fps=fps, frame_width=fw, fixtures_dir=fixtures)
398
399
400     def test_compare_expected_detects_changed_float(tmp_path):
401         """A float field changed by > 1e-6 is a mismatch (characterization catches behaviour
drift)."""
402         import copy
403         snap = _good_snapshot(tmp_path)
404         other = copy.deepcopy(snap)
405         other["score"]["f1"] = snap["score"]["f1"] - 1e-3
406         assert gf.compare_expected(other, snap), "a >1e-6 float change must be a non-empty

```

```

mismatch list"
407
408
409     def test_compare_expected_detects_changed_int(tmp_path):
410         """A changed int/structural field is a mismatch."""
411         import copy
412         snap = _good_snapshot(tmp_path)
413         other = copy.deepcopy(snap)
414         other["score"]["true_positives"] = snap["score"]["true_positives"] + 1
415         assert gf.compare_expected(other, snap)
416
417
418     def test_compare_expected_detects_candidate_reorder(tmp_path):
419         """Reordering candidates is a mismatch (order is structural ? first-vs-second window
matters)."""
420         import copy
421         # Use a 2-candidate scenario so a reorder is observable.
422         spec = gf.builtin_specs()[1] # multi_rally_with_deadtime
423         rows = gf.make_synthetic_trajectory(spec)
424         traj = tmp_path / "src_trajectory.csv"
425         labels = tmp_path / "src_labels.csv"
426         gf.write_trajectory_csv(rows, traj)
427         gf.write_labels_csv(spec.gts, labels)
428         snap = gf.freeze_real_fixture(
429             "scenario", traj, labels, slug="fx_reord001",
430             license="owned", minors_free=True, owned=True,
431             fps=spec.fps, frame_width=spec.frame_width, fixtures_dir=tmp_path / "fixtures")
432         assert len(snap["candidates"]) >= 2
433         other = copy.deepcopy(snap)
434         other["candidates"] = list(reversed(other["candidates"]))
435         assert gf.compare_expected(other, snap)
436
437
438     def test_compare_expected_detects_extra_top_level_key(tmp_path):
439         """red-team #5: an EXTRA top-level key on one side is a mismatch (key-set
equality)."""
440         import copy
441         snap = _good_snapshot(tmp_path)
442         other = copy.deepcopy(snap)
443         other["unexpected_field"] = 1
444         mism = gf.compare_expected(other, snap)
445         assert mism and any("top-level" in m for m in mism), mism
446
447
448     def test_compare_expected_detects_extra_score_key(tmp_path):
449         """red-team #5: an EXTRA key inside the score block is a mismatch (key-set
equality)."""
450         import copy
451         snap = _good_snapshot(tmp_path)
452         other = copy.deepcopy(snap)
453         other["score"]["bogus_metric"] = 0.0
454         mism = gf.compare_expected(other, snap)
455         assert mism and any(m.startswith("score:") for m in mism), mism
456
457
458     def test_scan_forbidden_raises_on_forbidden_key():
459         """red-team #10: the FORBIDDEN_KEYS branch raises (a person/audio/pose stream in the
text)."""
460         with pytest.raises(gf.RefusalError) as ei:
461             gf._scan_forbidden('{ "person": [1,2,3]}', where="x", source_paths=[])
462         assert "forbidden key" in str(ei.value)
463
464
465     def test_scan_forbidden_raises_on_md5():
466         """red-team #10: the MD5 branch raises (a 32-hex video_id shape is an attribution

```

```

back-channel)."""
467     md5 = "0123456789abcdef0123456789abcdef"
468     with pytest.raises(gf.RefusalError) as ei:
469         gf._scan_forbidden({'slug': "%s"} % md5, where="x", source_paths=[])
470     assert "MD5" in str(ei.value)
471
472
473     def test_refresh_snapshot_no_target_raises(tmp_path):
474         """red-team #10: refresh_snapshot with neither slug nor all raises ValueError."""
475         with pytest.raises(ValueError):
476             gf.refresh_snapshot(fixtures_dir=tmp_path)
477
478
479     def test_refresh_snapshot_cli_requires_reason(capsys):
480         """red-team #6: --refresh-snapshot --all WITHOUT --reason is a CLI error (argparse
481         exits 2)."""
482         with pytest.raises(SystemExit) as ei:
483             gf.main(["--refresh-snapshot", "--all"])
484         assert ei.value.code == 2 # argparse ap.error exit code
485         err = capsys.readouterr().err
486         assert "--reason" in err
487
488     def test_load_fixture_guards_provenance_schema_independently(tmp_path):
489         """red-team #7: a provenance schema_version > CURRENT (with expected sv in-range)
490         still raises ?
491         the two blocks are guarded INDEPENDENTLY, not masked by each other."""
492         fixtures = tmp_path / "fixtures"
493         traj, labels, fps, fw, _ = _write_source(tmp_path)
494         expected = gf.freeze_real_fixture(
495             "scenario", traj, labels, slug="fx_sv000001",
496             license="owned", minors_free=True, owned=True,
497             fps=fps, frame_width=fw, fixtures_dir=fixtures)
498         slug = expected["slug"]
499         # expected.json stays at the current schema; bump ONLY provenance to a future
500         version.
501         prov = gf._read_json(fixtures / slug / "provenance.json")
502         prov["schema_version"] = 99
503         gf._write_json(fixtures / slug / "provenance.json", prov)
504         with pytest.raises(ValueError) as ei:
505             gf.load_fixture(slug, fixtures_dir=fixtures)
506         assert "provenance schema_version 99" in str(ei.value)
507

```

15. assistant (2026-06-18T07:55:07.053Z)

Strong coverage on golden_fixtures. Now the ablation and experiment tests.

16. user (2026-06-18T07:55:08.553Z)

```

1     """GPU-free tests for the LOSO/LOGO ablation runner (`backend.eval.ablation`).
2
3     Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5     - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise
6     signal is
7     *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns /
8     zeroes
9     the gate knob without mutating ``full``.
10    - the structural-guard exemption ? the SAME signal marked ``structural=True`` is
11    reported
12    (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no
13    lift".
14    - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
15    stronger

```



```

11         than the stats support (the no-over-claim contract).
12     - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
13     - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
14         reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
15         cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16     """
17     from __future__ import annotations
18
19     import glob
20     import json
21     import os
22
23     import pytest
24
25     from backend.eval import ablation
26     from backend.eval.ablation import (
27         AblationConfig,
28         AblationReport,
29         Signal,
30         ablate_one,
31         run_ablation,
32     )
33     from backend.eval.experiment import Variant, VideoCandidates
34
35
36     # ----- synthetic
fixtures
37
38
39     def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
40         """``n`` videos with a ``net_crossings`` gate signal.
41
42         ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
``min_crossings=2`` gate
43         perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
44         has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
45         videos: list[VideoCandidates] = []
46         for k in range(n):
47             intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
48             junk_val = 5.0 if not separating else 0.0
49             feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
50             gts = [(0.0, 10.0), (20.0, 30.0)]
51             videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
52         return videos
53
54
55     def _learned_videos(n: int = 6) -> list[VideoCandidates]:
56         """``n`` videos where a PERSON feature (``players_in_court``) cleanly separates
rallies from
57         junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
58         ablation must show person as the load-bearing group and the others as washes ? a
deterministic
59         LOGO/LOSO correctness fixture (mirrors ``test_fusion_compare``'s construction)."""
60         videos: list[VideoCandidates] = []
61         for k in range(n):
62             intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
63             feats: dict[str, list[float]] = {}
64             for fn in ablation.SHUTTLE_COLUMNS:
65                 feats[fn] = [1.0, 1.0, 1.0, 1.0] # constant ? non-

```

```

separating
66         for fn in ablation.PERSON_COLUMNS:
67             feats[fn] = [0.0, 0.0, 0.0, 0.0]
68             feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0]          # the separating signal
69             feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
70         for fn in ablation.AUDIO_COLUMNS:
71             feats[fn] = [0.0, 0.0, 0.0, 0.0]
72             gts = [(0.0, 10.0), (20.0, 30.0)]
73             videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
74         return videos
75
76
77         # ----- LOSO
correctness
78
79
80     def test_separating_gate_earns_keep():
81         """A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing
it
82         drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""
83         videos = _gate_videos(6, separating=True)
84         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
85         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
86                               granularity="signal")
87         row = ablate_one(videos, cfg, sig)
88         assert row.delta_f1 > 0.0
89         assert row.ci_excludes_zero is True
90         assert row.sign_wins == 6 and row.sign_losses == 0
91         assert row.sign_p_value <= 0.05
92         assert row.verdict == ablation.EARNS_KEEP
93         assert row.structural_protected is False
94         # C4: the gate cuts over-segmentation (more junk segments without it).
95         assert row.seg_ratio_full < row.seg_ratio_ablated
96
97
98     def test_noop_gate_is_inert():
99         """A gate whose threshold removes nothing (all windows pass) is a wash ? INERT,
never a
100         stronger claim. Honest: a single wash at n=6 is a candidate, not 'proven
useless'."""
101         videos = _gate_videos(6, separating=False)
102         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
103         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
104                               granularity="signal")
105         row = ablate_one(videos, cfg, sig)
106         assert abs(row.delta_f1) < 1e-9
107         assert row.ci_excludes_zero is False
108         assert row.verdict == ablation.INERT
109
110
111     def test_ablated_variant_does_not_mutate_full():
112         """Building the ablated variant must derive a new Variant, never mutate ``full`` in
place."""
113         full = Variant(name="full", feature_names=["a", "b", "c"], min_crossings=2)
114         before = full.to_dict()
115         ablated = ablation._ablated_variant(full, ("b",), "min_crossings")
116         assert ablated.feature_names == ["a", "c"]
117         assert ablated.min_crossings == 0
118         assert full.to_dict() == before          # full untouched
119         assert full.feature_names == ["a", "b", "c"]
120
121
122     def test_unknown_gate_knob_raises():
123         full = Variant(name="full", feature_names=["a"])

```

```

124         with pytest.raises(ValueError):
125             ablation._ablated_variant(full, (), "min_bogus")
126
127
128     # ----- LOGO
correctness
129
130
131     def test_logo_collapses_groups():
132         """LOGO (`granularity='group'`) ablates a whole producer's columns at once: the
default
133         5-signal LOSO inventory collapses to 4 producer groups, and the union of columns is
dropped."""
134         groups = ablation._collapse_to_groups(ablation.default_signals())
135         names = [g.name for g in groups]
136         assert names == ["shuttle", "gate_a", "person", "audio"]
137         shuttle = next(g for g in groups if g.name == "shuttle")
138         # the shuttle group unions duration + the velocity block
139         assert set(shuttle.columns) == {"duration", "peak_velocity", "mean_velocity",
"active_ratio"}
140         gate = next(g for g in groups if g.name == "gate_a")
141         assert gate.gate_knob == "min_crossings"
142         assert gate.structural is True # structural flag OR-ed up
143
144
145     def test_logo_finds_separating_group():
146         """On the learned fixture where PERSON cleanly separates, the LOGO person row must
be the
147         load-bearing one (largest positive marginal ?F1) and the constant groups must be
~washes."""
148         videos = _learned_videos(6)
149         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
150                             granularity="group")
151         report = run_ablation(videos, cfg)
152         rows = {r.signal: r for r in report.rows}
153         assert set(rows) == {"shuttle", "gate_a", "person", "audio"}
154         top = report.sorted_rows()[0]
155         assert top.signal == "person"
156         assert rows["person"].delta_f1 > 0.0
157         # the constant audio group carries no information ? marginal ? 0
158         assert abs(rows["audio"].delta_f1) < 0.05
159
160
161     def test_run_ablation_both_granularities():
162         """LOSO yields one row per signal; LOGO yields one per group; both honest-LOVO on a
learned
163         full set."""
164         videos = _learned_videos(6)
165         loso = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
166                                                     granularity="signal"))
167         logo = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
168                                                     granularity="group"))
169
170         assert len(loso.rows) == len(ablation.default_signals()) # 5 LOSO rows
171         assert len(logo.rows) == 4 # 4 producer groups
172         assert loso.honest_lovo is True and logo.honest_lovo is True
173
174
175
176     def test_run_ablation_rejects_bad_granularity():
177         videos = _learned_videos(2)
178         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
179                             granularity="bogus")

```

```

180         with pytest.raises(ValueError):
181             run_ablation(videos, cfg)
182
183
184     def test_run_ablation_needs_a_video():
185         cfg = AblationConfig(full=ablation.learned_full_variant(),
186                               signals=ablation.default_signals())
187         from backend.eval.experiment import ExperimentError
188         with pytest.raises(ExperimentError):
189             run_ablation([], cfg)
190
191     # ----- structural-guard
192     exemption
193
194     def test_structural_guard_never_demoted():
195         """A structural guard with the SAME measured ? as a non-structural one that EARNS
196         its keep
197         must report verdict ``structural_protected`` (never auto-demoted), but record the
198         honest ?."""
199         videos = _gate_videos(6, separating=True)
200         non = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
201         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
202         full = Variant(name="full", min_crossings=2)
203         cfg = AblationConfig(full=full, signals=[non], granularity="signal")
204         r_non = ablate_one(videos, cfg, non)
205         r_struct = ablate_one(videos, cfg, struct)
206         # identical honest measurement?
207         assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
208         assert (r_non.sign_wins, r_non.sign_losses) == (r_struct.sign_wins,
209         r_struct.sign_losses)
210         # ?but the verdict differs: non-structural earns keep, structural is protected.
211         assert r_non.verdict == ablation.EARNS_KEEP
212         assert r_struct.verdict == ablation.STRUCTURAL_PROTECTED
213         assert r_struct.structural_protected is True
214
215     def test_structural_guard_protected_even_when_inert():
216         """A structural guard that is a measured wash is STILL reported as
217         ``structural_protected``
218         (never flagged dead weight on 'no measured lift')."""
219         videos = _gate_videos(6, separating=False)
220         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
221         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[struct],
222                               granularity="signal")
223         row = ablate_one(videos, cfg, struct)
224         assert abs(row.delta_f1) < 1e-9 # honestly a wash
225         assert row.verdict == ablation.STRUCTURAL_PROTECTED
226         assert row.structural_protected is True
227
228     def test_default_gate_a_is_structural():
229         """The shipped Gate-A signal must carry ``structural=True`` (the held-shuttle FP
230         guard)."""
231         sig = next(s for s in ablation.default_signals() if s.group == "gate_a")
232         assert sig.structural is True
233         assert sig.gate_knob == "min_crossings"
234
235     # ----- honest-
236     stats fields
237
238     def test_row_carries_full_honest_evidence():

```

```

237     """Every row ships the honest evidence: n, CI bounds, the sign test, and a verdict
in the
238     closed vocabulary (no over-claim)."""
239     videos = _learned_videos(6)
240     cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
241                         granularity="group")
242     report = run_ablation(videos, cfg)
243     valid = {ablation.EARNS_KEEP, ablation.SUGGESTIVE, ablation.INERT,
244             ablation.STRUCTURAL_PROTECTED}
245     for r in report.rows:
246         d = r.as_dict()
247         assert {"delta_f1", "ci_low", "ci_high", "ci_excludes_zero", "sign_test", "n",
248               "verdict"} <= set(d)
249         assert d["n"] == 6
250         assert d["sign_test"]["wins"] + d["sign_test"]["losses"] <= 6
251         # CI brackets the mean ( $\pm$ fp) ? never a bare mean detached from its interval.
252         assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
253         assert r.verdict in valid
254
255
256     def test_classify_verdicts_cover_all_branches():
257         """`_classify` maps evidence ? the closed verdict vocabulary, honestly:
258         structural always protected; CI-clears-0 + sign-agrees ? earns_keep; leans-positive
but
259         CI-spans-0 ? suggestive; a wash ? inert."""
260         # structural always protected, regardless of evidence
261         assert ablation._classify(True, True, 6, 0, 0.01, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
262         assert ablation._classify(True, False, 0, 6, 1.0, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
263         # earns keep: CI clears 0 AND sign agrees
264         assert ablation._classify(False, True, 6, 0, 0.03, 0.05) == (ablation.EARNS_KEEP,
False)
265         # suggestive: leans positive (more wins) but CI spans 0 / not significant
266         assert ablation._classify(False, False, 4, 2, 0.30, 0.05) == (ablation.SUGGESTIVE,
False)
267         # inert: not more wins than losses and CI spans 0
268         assert ablation._classify(False, False, 2, 4, 0.69, 0.05) == (ablation.INERT, False)
269         assert ablation._classify(False, False, 2, 2, 1.0, 0.05) == (ablation.INERT, False)
270
271
272     def test_no_over_claim_vocabulary():
273         """The verdict vocabulary must NOT contain 'irrelevant'/'useless' (attribution
contract)."""
274         forbidden = {"irrelevant", "useless", "dead", "remove"}
275         for v in ablation.VERDICT_LABEL.values():
276             assert not any(bad in v.lower() for bad in forbidden)
277
278
279     def test_report_round_trips_json():
280         """The structured report is JSON-able and sorted by ?F1 desc (contribution
ranking)."""
281         videos = _learned_videos(6)
282         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
283                                                     granularity="group"))
284         blob = json.dumps(report.as_dict()) # must not raise
285         back = json.loads(blob)
286         deltas = [r["delta_f1"] for r in back["rows"]]
287         assert deltas == sorted(deltas, reverse=True) # sorted desc
288         assert back["num_videos"] == 6
289         assert back["label_set_hash"] # provenance present
290
291
292

```

```

293     def test_seed_is_deterministic():
294         """Two runs with the same seed produce identical CIs (deterministic bootstrap)."""
295         videos = _learned_videos(6)
296         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
297                             granularity="group", seed=7)
298         r1 = run_ablation(videos, cfg)
299         r2 = run_ablation(videos, cfg)
300         assert [r.ci_low for r in r1.rows] == [r.ci_low for r in r2.rows]
301         assert [r.ci_high for r in r1.rows] == [r.ci_high for r in r2.rows]
302
303
304     # ----- table +
PDF
305
306
307     def test_format_table_is_sorted_and_readable():
308         videos = _learned_videos(6)
309         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
310                                                         granularity="group"))
311
312         out = ablation.format_table(report)
313         assert "Ablation report" in out
314         assert "?F1(full?ablated)" in out
315         assert "verdict" in out
316         # the highest-?F1 signal name appears before the lowest in the rendered order
317         top = report.sorted_rows()[0].signal
318         bottom = report.sorted_rows()[-1].signal
319         assert out.index(top) < out.index(bottom)
320
321
322     def test_export_pdf_is_valid_nontrivial(tmp_path):
323         """A valid PDF with at least one page is produced, and it contains the verdict
legend."""
324         pytest.importorskip("reportlab")
325         pypdf = pytest.importorskip("pypdf")
326         videos = _learned_videos(6)
327         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
328                                                         granularity="group"))
329
330         out = str(tmp_path / "ablation.pdf")
331         ablation.export_pdf(report, out)
332         assert os.path.isfile(out)
333         assert os.path.getsize(out) > 1000 # non-trivial
334         reader = pypdf.PdfReader(out)
335         assert len(reader.pages) > 0
336         text = "".join(p.extract_text() for p in reader.pages)
337         assert "Ablation" in text
338         assert "clears-bar" in text # the verdict key rendered
339
340
341     # ----- corpus loader
(no GPU)
342
343
344     def test_load_videos_tolerates_missing_audio(tmp_path):
345         """An older feature JSON without an ``audio`` block still loads ? audio columns
default to
346         0.0 (loader never crashes on a pre-audio JSON)."""
347         cand = {"start": 0.0, "end": 10.0,
348                 "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
349                 "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
350                 "label": 1}
351         data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
352                 "candidates": [cand], "gts": [[0.0, 10.0]]}

```

```

353     with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
354         json.dump(data, f)
355     videos = ablation.load_videos(str(tmp_path))
356     assert len(videos) == 1
357     assert videos[0].features["audio_hit_count"] == [0.0]          # defaulted, not
crashed
358
359
360 def _write_windowing_only(d: str, name: str) -> None:
361     """Write a LEANER velocity-windowing/distill ``*_golden_features.json`` ? the schema
the
362     R3/R4 corpus-growth path persists for clips that never went through fusion feature
363     extraction (``start``/``end`` + velocity stats, NO shuttle/person/audio cue dicts).
The
364     ablation loader must SKIP these, never crash on ``c["shuttle"]``.
365     cands = [{"start": 0.0, "end": 5.0, "active_frame_count": 90, "chunk_index": 0,
366              "mean_velocity": 2.0, "peak_velocity": 9.0, "track_id_count": 1}]
367     data = {"name": name, "fps": 30.0, "frame_width": 1920.0,
368            "candidates": cands, "gts": [[0.0, 5.0]]}
369     with open(os.path.join(d, f"{name}_golden_features.json"), "w", encoding="utf-8") as
f:
370         json.dump(data, f)
371
372
373 def test_load_videos_skips_windowing_only_schema(tmp_path):
374     """Mixed-corpus regression (the n=11 ``KeyError: 'shuttle'``): full-fusion files
load; the
375     leaner velocity-windowing files (the 5 newer real clips) are SKIPPED rather than
crashed on,
376     and the litmus path runs end-to-end over the filtered ablatable subset.
377     d = str(tmp_path)
378     _write_learned_corpus(d, n=6)          # 6 full-fusion files
(v0..v5)
379     for name in ("mahadevpura_1", "GX010128", "Badminton_BXH_2"): # 3 leaner real-clip
names
380         _write_windowing_only(d, name)
381     # only the fusion-schema files load ? the leaner ones never crash the loader
382     videos = ablation.load_videos(d)
383     assert {v.name for v in videos} == {f"v{k}" for k in range(6)}
384     # ?and the litmus path (which reads the cue columns) runs cleanly on the mixed
corpus
385     report = run_ablation(videos, ablation.litmus_config())
386     assert [r.signal for r in report.rows] == ["gate_a"]
387     assert report.num_videos == 6
388
389
390 # ----- the
golden litmus
391
392     GOLDEN_DIR = os.path.join(
393         os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output")
394
395
396 def _golden_present() -> bool:
397     """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the
shuttle/person cue
398     columns). Counts only ablatable files ? leaner velocity-windowing files are skipped
by
399     ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus
(the n=11
400     mixed-schema bug).
401     n = 0
402     for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
403         try:
404             with open(p, encoding="utf-8") as f:

```

```

405         cands = json.load(f).get("candidates") or []
406     except (OSError, ValueError):
407         continue
408     if ablation._is_fusion_schema(cands):
409         n += 1
410     return n >= 6
411
412
413     @pytest.mark.skipif(not _golden_present(),
414                         reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
415     def test_litmus_reproduces_gate_a():
416         """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
417         result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
418         This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
419         where the golden substrate is absent; runs on the owner box where it is present."""
420         videos = ablation.load_videos(GOLDEN_DIR)
421         assert len(videos) == 6
422         cfg = ablation.litmus_config()
423         report = run_ablation(videos, cfg)
424         assert len(report.rows) == 1
425         row = report.rows[0]
426         assert row.signal == "gate_a"
427         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
428         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
429         assert row.sign_wins == 6 and row.sign_losses == 0
430         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
431         assert row.ci_low == pytest.approx(0.042, abs=0.01)
432         assert row.ci_high == pytest.approx(0.104, abs=0.01)
433         assert row.ci_excludes_zero is True
434         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
435         assert row.verdict == ablation.STRUCTURAL_PROTECTED
436         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
437         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
438         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
439
440
441     @pytest.mark.skipif(not _golden_present(),
442                         reason="golden output/*_golden_features.json absent")
443     def test_litmus_report_is_well_formed():
444         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
445         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
446         assert isinstance(report, AblationReport)
447         assert report.num_videos == 6
448         assert report.full_set_signals == ["gate_a"]
449         assert report.label_set_hash
450         out = ablation.format_table(report)
451         assert "gate_a" in out
452
453
454     # ----- edge
cases
455
456
457     def test_min_players_gate_knob_zeroed():
458         """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
459         full = Variant(name="full", min_players=2, feature_names=["a"])
460         ablated = ablation.ablated_variant(full, (), "min_players")
461         assert ablated.min_players == 0
462         assert ablated.feature_names == ["a"]          # columns untouched
463

```



```

464
465 def test_ablated_label_falls_back_to_gate_knob():
466     """With no columns, the ablated variant name carries the gate-knob label, not
467     '?'."""
468     full = Variant(name="full", min_crossings=2)
469     ablated = ablation._ablated_variant(full, (), "min_crossings")
470     assert "min_crossings" in ablated.name
471
472 # -----
473 CLI
474
475 def _write_learned_corpus(d: str, n: int = 6) -> None:
476     for k in range(n):
477         cands = []
478         for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
479 ((50.0, 51.0), 0)]:
480             shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
481             person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
482             person["players_in_court"] = 4.0 if pos else 0.0
483             person["in_court_ratio"] = 1.0 if pos else 0.0
484             audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
485             cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
486 "audio": audio, "label": pos})
487             data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
488 "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
489             with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
490 as f:
491                 json.dump(data, f)
492
493 def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
494     """`main()` runs the LOGO ablation and writes both a PDF and the structured
495     JSON."""
496     pytest.importorskip("reportlab")
497     pytest.importorskip("pypdf")
498     d = str(tmp_path / "feat")
499     os.makedirs(d)
500     _write_learned_corpus(d)
501     pdf = str(tmp_path / "out.pdf")
502     js = str(tmp_path / "out.json")
503     monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
504 "group",
505 "--pdf", pdf, "--json", js])
506     ablation.main()
507     out = capsys.readouterr().out
508     assert "Ablation report" in out
509     assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
510     assert os.path.isfile(js)
511     report = json.loads(open(js, encoding="utf-8").read())
512     assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
513 "audio"}
514
515 def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):
516     """`--litmus` runs the static Gate-A config (single structural row)."""
517     d = str(tmp_path / "feat")
518     os.makedirs(d)
519     _write_learned_corpus(d)
520     monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
521     ablation.main()
522     out = capsys.readouterr().out
523     assert "gate_a" in out
524     assert "static" in out
525     # litmus full set is the static

```

```

gate variant
522
523
524     def test_cli_main_needs_two_videos(tmp_path, monkeypatch):
525         d = str(tmp_path / "feat")
526         os.makedirs(d)
527         _write_learned_corpus(d, n=1)
528         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d])
529         with pytest.raises(SystemExit):
530             ablation.main()
531

```

17. assistant (2026-06-18T07:55:11.862Z)

Excellent ? ablation has very strong coverage including PDF export (via pypdf), CLI, and `load\_videos`. Now let me check the experiment and rally_seg_eval test files, and confirm whether `\_is\_fusion\_schema` is duplicated (the docstring says it was consolidated).

18. user (2026-06-18T07:55:13.468Z)

```

backend/eval/ablation.py:57:from backend.eval.experiment import Variant, VideoCandidates,
_is_fusion_schema
backend/eval/ablation.py:386:def load_videos(features_dir: str) -> List[VideoCandidates]:
backend/eval/ablation.py:394:     ablatable subset cleanly (see ``_is_fusion_schema``)."""
backend/eval/ablation.py:400:         if not _is_fusion_schema(cands):
backend/eval/experiment.py:180:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) ->
bool:
backend/eval/fusion_audio.py:23:from backend.eval.experiment import _is_fusion_schema
backend/eval/fusion_audio.py:56:def load_videos_with_audio(features_dir: str) ->
List[experiment.VideoCandidates]:
backend/eval/fusion_audio.py:61:     crashed on, so a mixed corpus loads the fusion subset
cleanly (see ``_is_fusion_schema``).
backend/eval/fusion_audio.py:69:         if not _is_fusion_schema(cands):
backend/eval/fusion_compare.py:23:from backend.eval.experiment import _is_fusion_schema
backend/eval/fusion_compare.py:39:def load_videos(features_dir: str) ->
List[experiment.VideoCandidates]:
backend/eval/fusion_compare.py:46:     fusion subset cleanly (see ``_is_fusion_schema``)."""
backend/eval/fusion_compare.py:52:         if not _is_fusion_schema(cands):

```

19. user (2026-06-18T07:55:15.061Z)

```

116 backend/eval/fusion_compare.py
146 backend/eval/fusion_audio.py
195 backend/eval/fusion_golden.py
457 total
=== tests for experiment/rally_seg ===
244 tests/test_experiment.py
62 tests/test_experiment_honest.py
93 tests/test_rally_seg_eval.py
399 total

```

20. assistant (2026-06-18T07:55:19.012Z)

`\_is\_fusion\_schema` is already consolidated (one definition in experiment.py, imported by the three loaders). Now let me look at the three `load\_videos` bodies to assess duplication, and read the experiment + rally_seg tests.

21. user (2026-06-18T07:55:19.992Z)

```

39     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
40         """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a VideoCandidates
41         (intervals + the 10 feature columns + golden gts). The harness derives labels from
gts
42         itself.

```

```

43
44     Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
45     corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads
the
46     fusion subset cleanly (see ``_is_fusion_schema``)."""
47     videos: List[experiment.VideoCandidates] = []
48     for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
49         with open(path, encoding="utf-8") as f:
50             d = json.load(f)
51             cands = d["candidates"]
52             if not _is_fusion_schema(cands):
53                 logger.info("fusion_compare.load_videos: skipping %s ? windowing-only schema
"
54                             "(no shuttle/person cue dicts), not a fusion-feature file",
55                             os.path.basename(path))
56                 continue
57             intervals = [(float(c["start"]), float(c["end"])) for c in cands]
58             features: Dict[str, List[float]] = {}
59             for fn in SHUTTLE_FEATURE_NAMES:
60                 features[fn] = [float(c["shuttle"][fn]) for c in cands]
61             for fn in PERSON_FEATURE_NAMES:
62                 features[fn] = [float(c["person"][fn]) for c in cands]
63             gts = [(float(a), float(b)) for a, b in d["gts"]]
64             videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
65                                                         features=features, gts=gts))
66     return videos
67
68
69     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
70                 threshold: float = 0.5) -> Dict[str, Any]:
71         shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
72                                     threshold=threshold)
73         fusion = experiment.Variant(name="shuttle_person_fusion",

```

22. user (2026-06-18T07:55:23.978Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5     window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6     and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7     of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8     (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10    Run (no GPU):
11        python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12    """
13    from __future__ import annotations
14
15    import argparse
16    import glob
17    import json
18    import logging
19    import os
20    from typing import Any, Dict, List

```

```

21
22     from backend.eval import experiment
23     from backend.eval.experiment import _is_fusion_schema
24     from backend.eval.fusion_compare import delta_stats
25     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
26     from backend.pipeline.detectors import audio_features as af
27     from backend.pipeline.audio.hit_detector import detect_hits_in_video
28
29     logger = logging.getLogger(__name__)
30
31
32     def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
33         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
34         with open(manifest_path, encoding="utf-8") as f:
35             specs = json.load(f)
36             written: List[str] = []
37             for spec in specs:
38                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
39                 if not os.path.isfile(jp):
40                     print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping")
41                     continue
42                 video = _derive_video_path(spec)
43                 print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
flush=True)
44                 hits = detect_hits_in_video(video, k=k)
45                 with open(jp, encoding="utf-8") as f:
46                     data = json.load(f)
47                     for c in data["candidates"]:
48                         c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
49                 with open(jp, "w", encoding="utf-8") as f:
50                     json.dump(data, f, indent=2)
51                 print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows", flush=True)
52                 written.append(jp)
53             return written
54
55
56     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
57         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
58         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
59
60         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are
SKIPPED, not
61         crashed on, so a mixed corpus loads the fusion subset cleanly (see
``_is_fusion_schema``).
62         The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that
simply
63         hasn't been ``--augment``-ed still raises rather than being silently dropped."""
64         videos: List[experiment.VideoCandidates] = []
65         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
66             with open(path, encoding="utf-8") as f:
67                 d = json.load(f)
68                 cands = d["candidates"]
69                 if not _is_fusion_schema(cands):
70                     logger.info("fusion_audio.load_videos_with_audio: skipping %s ? windowing-
only "
71                                "schema (no shuttle/person cue dicts), not a fusion-feature
file",
72                                os.path.basename(path))
73                     continue
74                 if "audio" not in cands[0]:

```

```

75         raise SystemExit(f"{path} has no audio features ? run `--augment` first")
76     intervals = [(float(c["start"]), float(c["end"])) for c in candsets]
77     features: Dict[str, List[float]] = {}
78     for fn in SHUTTLE_FEATURE_NAMES:
79         features[fn] = [float(c["shuttle"][fn]) for c in candsets]
80     for fn in PERSON_FEATURE_NAMES:
81         features[fn] = [float(c["person"][fn]) for c in candsets]
82     for fn in af.AUDIO_FEATURE_NAMES:
83         features[fn] = [float(c["audio"][fn]) for c in candsets]
84     gts = [(float(a), float(b)) for a, b in d["gts"]]
85     videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
86                                             features=features, gts=gts))
87     return videos
88
89
90     def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
91                          threshold: float = 0.5) -> Dict[str, Any]:
92         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
93         LOVO)."""
94         person = experiment.Variant(
95             name="shuttle_person",
96             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
97             threshold=threshold)

```

23. assistant (2026-06-18T07:55:28.637Z)

Now I can see the duplication pattern clearly. The three `load\_videos` functions (ablation.load_videos, fusion_compare.load_videos, fusion_audio.load_videos_with_audio) share an identical glob/load/skip/build loop. Let me check the experiment and rally_seg test files, and check coverage of fusion loaders.

24. user (2026-06-18T07:55:29.930Z)

```

1     """Tests for backend.eval.rally_seg_eval ? the pure scoring/aggregation core (no golden
2     files,
3     no trajectory I/O). The windowing + trajectory parsing it composes are tested in their
4     own modules.
5     """
6     import pytest
7
8     from backend.eval.rally_seg_eval import aggregate, compare, evaluate, score_intervals,
9     windows_for
10    from backend.eval.windowing import WindowingPreset
11
12    GTS = [(0.0, 10.0), (20.0, 30.0), (40.0, 50.0)]
13    # A permissive preset so a short synthetic trajectory still forms a window.
14    _PRESET = WindowingPreset(name="t", velocity_thresh=0.02, merge_gap=2.0,
15    min_window_duration=0.5)
16
17
18    def _write_oscillating_traj(path, fps=30, secs=4, width=1280):
19        """A fast shuttle oscillating smoothly across the net (x=width/2) via a triangle
20        wave ?
21        continuous high velocity (active) + many net crossings ? one gated-survivable
22        window."""
23        lo, hi, period = 200, width - 200, 20 # frames per full oscillation
24        rows = ["Frame,Visibility,X,Y"]
25        for i in range(int(fps * secs)):
26            phase = (i % period) / period # 0..1
27            tri = 2 * phase if phase < 0.5 else 2 * (1 - phase) # triangle 0?1?0
28            rows.append(f"{i},1,{lo + (hi - lo) * tri:.1f},300")
29        path.write_text("\n".join(rows))
30        return str(path)

```

```

27     def test_score_intervals_perfect():
28         row = score_intervals(GTS, GTS)
29         assert row["f1"] == pytest.approx(1.0)
30         assert row["f1@0.25"] == pytest.approx(1.0)
31         assert row["merge_rate"] == 0.0 and row["split_rate"] == 0.0
32         assert row["merges"] == 0 and row["num_pred"] == 3 and row["num_gt"] == 3
33
34
35     def test_score_intervals_surfaces_over_merge():
36         # One mega-window swallowing two rallies: tIoU-F1 is hurt AND merge_rate makes it
explicit.
37         preds = [(0.0, 30.0), (40.0, 50.0)]
38         row = score_intervals(preds, GTS)
39         assert row["f1"] < 1.0
40         assert row["merges"] == 1
41         assert row["merge_rate"] == pytest.approx(2 / 3)
42         assert row["segment_count_ratio"] == pytest.approx(2 / 3)
43
44
45     def test_aggregate_means_across_videos_with_ci():
46         rows = [score_intervals(GTS, GTS), # perfect
47                 score_intervals([(0.0, 30.0), (40.0, 50.0)], GTS)] # over-merged
48         agg = aggregate(rows)
49         assert agg["n"] == 2
50         # mean F1 sits between the two videos' F1s, and the CI block is present.
51         assert 0.0 < agg["f1"]["mean"] < 1.0
52         assert "ci_low" in agg["f1"] and "ci_high" in agg["f1"]
53         assert agg["merge_rate"]["mean"] == pytest.approx((0.0 + 2 / 3) / 2)
54
55
56     def test_aggregate_empty():
57         agg = aggregate([])
58         assert agg["n"] == 0 and agg["f1"] is None
59
60
61     # --- W2: net-crossings gate (Gate-A) on the windowing -----
62     #
63     def test_windows_for_netcross_gate(tmp_path):
64         pytest.importorskip("cv2") # rally_gate import path tolerance (no cv2 actually
needed)
65         video = {"trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
66                 "fps": 30.0, "frame_width": 1280.0}
67         ungated = windows_for(video, _PRESET, min_crossings=0)
68         assert len(ungated) >= 1 # a window forms
69         # The oscillating shuttle crosses the net many times ? survives a modest gate...
70         assert len(windows_for(video, _PRESET, min_crossings=2)) >= 1
71         # ...but an absurd crossing requirement that would gate EVERY window out instead
falls
72         # back to the ungated set (never-empty safeguard) ? the gate prunes, never zeroes.
73         assert windows_for(video, _PRESET, min_crossings=999) == ungated
74
75     def test_evaluate_threads_min_crossings(tmp_path):
76         video = {"name": "osc", "trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
77                 "fps": 30.0, "frame_width": 1280.0, "gts": [(0.0, 4.0)]}
78         res = evaluate([video], _PRESET, min_crossings=3)
79         assert res["min_crossings"] == 3 and res["aggregate"]["n"] == 1
80
81
82     # --- R1: compare() surfaces the net over-seg promotion guard
----- #
83     def test_compare_includes_over_seg_guard(tmp_path):
84         video = {"name": "osc", "trajectory": _write_oscillating_traj(tmp_path / "osc.csv"),
85                 "fps": 30.0, "frame_width": 1280.0, "gts": [(0.0, 4.0)]}
86         res = compare([video], _PRESET, _PRESET) # identical presets ? no over-seg change

```

```

87         guard = res["over_seg_guard"]
88         # the guard is a first-class part of the comparison output, with both verdicts
reported
89         assert set(["passes", "strict_passes", "base_net", "cand_net", "net_delta",
90                    "use_rates", "weight_merge", "weight_split"]).issubset(guard)
91         assert guard["use_rates"] is True and guard["weight_merge"] == 2.0
92         # identical windowing ? no over-seg movement ? both the net and strict rules pass
93         assert guard["passes"] is True and guard["strict_passes"] is True
94

```

25. assistant (2026-06-18T07:55:33.815Z)

rally_seg_eval: the pure scoring core (`score\_intervals`, `aggregate`, `evaluate`, `compare`, `windows\_for`) is well-tested, but the reporting layer (`format\_report`, `\_format\_guard`, `\_ci`) and the `main()` CLI (with its large `replace`-based preset wiring) have NO tests. Now the experiment tests.

26. user (2026-06-18T07:55:35.008Z)

```

1     """Tests for the experiment harness (backend.eval.experiment).
2
3     Pure/offline ? synthetic candidate rows, no GPU/WSL. The load-bearing test is the
4     **no-regression invariant** (`test_disabled_cues_byte_identical_to_control`): a variant
5     that merely carries a disabled cue must predict exactly what control predicts. That is
the
6     guarantee that lets new cues be merged freely (`EXPERIMENT_HARNESS.md` §6).
7     """
8     import json
9
10    import pytest
11
12    from backend.eval.classifier import LogisticRegression
13    from backend.eval.experiment import (
14        CONTROL,
15        ExperimentError,
16        Variant,
17        VideoCandidates,
18        _is_fusion_schema,
19        compare,
20        compare_unlabeled,
21        evaluate_variant,
22        load_video_candidates,
23        predict,
24        record_experiment,
25    )
26    from backend.infrastructure.database import Database
27
28
29    # A video whose third candidate is a held-shuttle false fire (net_crossings=0): it does
30    # not overlap any golden rally, so it is a negative by construction.
31    def _held_shuttle_video(name="v", gts=None):
32        return VideoCandidates(
33            name=name,
34            intervals=[(0.0, 10.0), (20.0, 30.0), (12.0, 13.0)],
35            features={"net_crossings": [4.0, 3.0, 0.0]},
36            gts=gts if gts is not None else [(0.0, 10.0), (20.0, 30.0)],
37        )
38
39
40    # Three videos where feature "x" perfectly separates rallies (x=5) from junk (x=0),
41    # so an honest LOVO learned variant should recover F1?1.0 on each held-out video.
42    def _separable_videos():
43        vids = []
44        for k in range(3):
45            base = 100.0 * k

```

```

46         vids.append(VideoCandidates(
47             name=f"sep{k}",
48             intervals=[(base + 0, base + 10), (base + 20, base + 30),
49                 (base + 40, base + 41), (base + 50, base + 51)],
50             features={"x": [5.0, 5.0, 0.0, 0.0]},
51             gts=[(base + 0, base + 10), (base + 20, base + 30)],
52         ))
53     return vids
54
55
56     # ----- golden-features
57     schema
58     def test_is_fusion_schema_distinguishes_lean_and_full():
59         """The shared schema predicate keeps full-fusion rows (shuttle+person, audio
60 optional) and
61 rejects the leaner velocity-windowing rows (no cue dicts); an empty candidate list
62 is not a
63 fusion file. Single source of truth for the three golden-feature loaders
64 (ablation.load_videos / fusion_compare.load_videos /
65 fusion_audio.load_videos_with_audio)."""
66     full = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}, "audio": {}}]
67     no_audio = [{"start": 0.0, "end": 1.0, "shuttle": {}, "person": {}}] #
68     audio optional
69     lean = [{"start": 0.0, "end": 1.0, "mean_velocity": 1.0, "track_id_count": 1}]
70     assert _is_fusion_schema(full) is True
71     assert _is_fusion_schema(no_audio) is True
72     assert _is_fusion_schema(lean) is False
73     assert _is_fusion_schema([]) is False
74
75     # ----- Variant
76
77     def test_variant_roundtrip_and_hash_stability():
78         v = Variant(name="gateA", min_crossings=2, cue_flags={"person": False})
79         assert Variant.from_dict(v.to_dict()) == v
80         assert v.spec_hash() == v.spec_hash() # stable
81         assert CONTROL.spec_hash() == Variant(name="control").spec_hash()
82         assert v.spec_hash() != CONTROL.spec_hash() # different recipe ?
83     different hash
84
85
86     def test_from_dict_ignores_unknown_keys():
87         v = Variant.from_dict({"name": "x", "min_crossings": 1, "bogus": 99})
88         assert v.name == "x" and v.min_crossings == 1
89
90
91     def test_is_learned():
92         assert not CONTROL.is_learned()
93         assert Variant(name="r", feature_names=["x"]).is_learned()
94         assert Variant(name="p", classifier_model="model.json").is_learned()
95
96     # ----- the no-regression invariant
97
98     def test_disabled_cues_byte_identical_to_control():
99         """A variant carrying only DISABLED cues must equal control, window-for-window."""
100         vc = _held_shuttle_video()
101         disabled = Variant(name="fusion_off", cue_flags={"person": False, "audio": False},
102             min_crossings=0, min_players=0)
103         assert predict(disabled, vc) == predict(CONTROL, vc)
104
105
106     def test_control_is_merged_candidates():
107         vc = VideoCandidates(name="v", intervals=[(0.0, 10.0), (10.5, 12.0), (40.0, 50.0)])

```



```

105         # merge_gap=1.0 stitches the 0.5s gap between the first two; the 28s gap stays
split.
106         assert predict(CONTROL, vc) == [(0.0, 12.0), (40.0, 50.0)]
107
108
109     # ----- decision gates
110
111     def test_gate_a_drops_low_crossing_window():
112         vc = _held_shuttle_video()
113         gate_a = Variant(name="gateA", min_crossings=2)
114         preds = predict(gate_a, vc)
115         assert (12.0, 13.0) not in preds # held-shuttle fire
removed
116         assert preds == [(0.0, 10.0), (20.0, 30.0)]
117         assert preds != predict(CONTROL, vc) # the gate actually
changed output
118
119
120     def test_gate_b_filters_on_players():
121         vc = VideoCandidates(
122             name="v", intervals=[(0.0, 10.0), (20.0, 30.0)],
123             features={"players_in_court": [2.0, 1.0]})
124         preds = predict(Variant(name="gateB", min_players=2), vc)
125         assert preds == [(0.0, 10.0)] # the 1-player window is
dropped
126
127
128     def test_gate_on_absent_feature_defaults_to_zero(caplog):
129         vc = VideoCandidates(name="v", intervals=[(0.0, 10.0)]) # no net_crossings
persisted
130         preds = predict(Variant(name="gateA", min_crossings=2), vc)
131         assert preds == [] # absent feature ? 0 ?
fails gate
132
133
134     # ----- evaluate_variant
135
136     def test_evaluate_static_variant_scores_vs_gts():
137         res = evaluate_variant([_held_shuttle_video()], CONTROL, iou=0.5)
138         pv = res["per_video"][0]
139         # control keeps the false fire: 2 TP, 1 FP ? precision 2/3, recall 1.0
140         assert pv["recall"] == pytest.approx(1.0)
141         assert pv["precision"] == pytest.approx(2 / 3)
142         assert res["aggregate"]["recall"] == pytest.approx(1.0)
143         assert not res["honest_lovo"]
144
145
146     def test_evaluate_learned_lovo_recovers_separable_signal():
147         variant = Variant(name="learned", feature_names=["x"], threshold=0.5)
148         res = evaluate_variant(_separable_videos(), variant, iou=0.5, honest=True)
149         assert res["honest_lovo"] is True
150         assert res["aggregate"]["f1"] == pytest.approx(1.0) # held-out F1 perfect on
separable data
151
152
153     def test_evaluate_learned_lovo_needs_two_videos():
154         variant = Variant(name="learned", feature_names=["x"])
155         with pytest.raises(ExperimentError):
156             evaluate_variant(_separable_videos()[:1], variant, honest=True)
157
158
159     def test_evaluate_requires_labels():
160         with pytest.raises(ExperimentError):
161             evaluate_variant([VideoCandidates(name="v", intervals=[(0.0, 1.0)], gts=None)],
CONTROL)

```

```

162
163
164 # ----- compare
165
166 def test_compare_gate_a_improves_precision():
167     videos = [_held_shuttle_video()]
168     res = compare(videos, CONTROL, Variant(name="gateA", min_crossings=2), iou=0.5)
169     assert res["variant_b"]["aggregate"]["precision"] == pytest.approx(1.0)
170     assert res["delta"]["precision"] > 0 # B (gate) beats A
(control) on precision
171     assert res["delta"]["f1"] > 0
172     assert res["label_set_hash"] # fingerprint present
173     assert res["num_videos"] == 1
174     assert len(res["per_video_delta"]) == 1
175
176
177 def test_compare_needs_a_video():
178     with pytest.raises(ExperimentError):
179         compare([], CONTROL, CONTROL)
180
181
182 # ----- compare_unlabeled
183
184 def test_compare_unlabeled_partitions_agreement():
185     vc = _held_shuttle_video(gts=None) # new footage, no labels
186     res = compare_unlabeled(vc, CONTROL, Variant(name="gateA", min_crossings=2))
187     assert res["num_a"] == 3 and res["num_b"] == 2
188     assert res["num_agreed"] == 2
189     assert res["a_only"] == [(12.0, 13.0)] # A fired, B suppressed
190     assert res["b_only"] == [] # B is a subset of A here
191     assert res["mean_agree_iou"] == pytest.approx(1.0)
192
193
194 def test_compare_unlabeled_learned_variant_needs_pinned_model():
195     vc = VideoCandidates(name="new", intervals=[(0.0, 10.0)], features={"x": [5.0]})
196     learned_no_model = Variant(name="learned", feature_names=["x"])
197     with pytest.raises(ExperimentError):
198         compare_unlabeled(vc, CONTROL, learned_no_model)
199
200
201 def test_compare_unlabeled_with_pinned_model(tmp_path):
202     # Train + persist a model, then use it as a pinned variant on an unlabeled video.
203     model = LogisticRegression().fit([[5.0], [5.0], [0.0], [0.0]], [1, 1, 0, 0], ["x"])
204     model_path = str(tmp_path / "m.json")
205     model.save(model_path)
206     vc = VideoCandidates(name="new", intervals=[(0.0, 10.0), (40.0, 41.0)],
207         features={"x": [5.0, 0.0]})
208     pinned = Variant(name="learned", feature_names=["x"], classifier_model=model_path)
209     res = compare_unlabeled(vc, CONTROL, pinned)
210     assert res["num_a"] == 2 # control keeps both
211     assert res["num_b"] == 1 # model drops the x=0
window
212     assert res["a_only"] == [(40.0, 41.0)]
213
214
215 # ----- record_experiment
216
217 def test_record_experiment_appends(tmp_path):
218     path = str(tmp_path / "leaderboard.json")
219     res = compare(_held_shuttle_video(), CONTROL, Variant(name="gateA",
min_crossings=2))
220     row1 = record_experiment(res, path, timestamp="2026-06-13T00:00:00Z")
221     row2 = record_experiment(res, path, timestamp="2026-06-13T01:00:00Z")
222     assert row1["variant_b"]["name"] == "gateA"
223     assert row1["timestamp"] != row2["timestamp"]

```

```

224         with open(path, encoding="utf-8") as f:
225             entries = json.load(f)
226             assert len(entries) == 2                                # appended, not
overwritten
227             assert entries[0]["delta"]["precision"] > 0
228
229
230     # -----
load_video_candidates
231
232     def test_load_video_candidates_from_db(tmp_path):
233         db = Database(str(tmp_path / "t.db"))
234         db.add_video("vid1", "/x.mp4", "ok", [])
235         db.add_candidate_segment("vid1", "wasb_hybrid", 0.0, 10.0,
236                                 stats={"net_crossings": 4, "peak_velocity": 0.3})
237         db.add_candidate_segment("vid1", "wasb_hybrid", 12.0, 13.0, stats={"net_crossings":
238                                     0})
239
240         vc = load_video_candidates(db, "vid1", detector="wasb_hybrid", gts=[(0.0, 10.0)],
241                                   feature_names=["net_crossings", "duration"])
242         assert vc.intervals == [(0.0, 10.0), (12.0, 13.0)]
243         assert vc.features["net_crossings"] == [4.0, 0.0]
244         assert vc.features["duration"] == [10.0, 1.0]                # derived from bounds
245         assert vc.gts == [(0.0, 10.0)]

```

27. assistant (2026-06-18T07:55:40.517Z)

experiment.py has strong, comprehensive coverage on all the public surface (predict, gates, evaluate_variant, compare, compare_unlabeled, record_experiment, load_video_candidates, _is_fusion_schema). Let me check test_experiment_honest.py and verify whether the fusion loaders themselves are tested (to assess the dedup proposal's test backing). Let me also check how `run\_all\_tests.py` runs and whether `reportlab`/`pypdf`/`numpy` are available here.

28. user (2026-06-18T07:55:41.437Z)

```

1     """Tests for the honest-reporting wiring in experiment.compare (C1 + C4, additive).
2
3     Pins: compare() ADDS a `honest` block by default (CIs + paired ?F1 sign test + F1@k +
4     segment-count ratio) WITHOUT changing the existing fields; `honest_stats=False` restores
5     the legacy shape; and record_experiment carries the honest ?F1.
6     """
7     from backend.eval.experiment import CONTROL, Variant, VideoCandidates, compare,
record_experiment
8
9     GATE = Variant(name="gateA", min_crossings=2)
10
11
12     def _vid(name, net_crossings):
13         # 3 candidates; the MIDDLE is a false fire (net_crossings 0) absent from the golden
set,
14         # so Gate-A (>=2) drops it: precision/F1 up, over-segmentation 1.5 -> 1.0.
15         return VideoCandidates(
16             name=name,
17             intervals=[(0.0, 10.0), (20.0, 30.0), (40.0, 50.0)],
18             features={"net_crossings": net_crossings},
19             gts=[(0.0, 10.0), (40.0, 50.0)],
20         )
21
22
23     def _corpus():
24         return [_vid("v1", [5.0, 0.0, 4.0]), _vid("v2", [3.0, 0.0, 6.0])]
25
26
27     def test_compare_adds_honest_block_without_touching_legacy_fields():

```

```

28     res = compare(_corpus(), CONTROL, GATE)
29     # legacy fields intact (no regression)
30     for k in ("iou", "label_set_hash", "num_videos", "variant_a", "variant_b", "delta",
31              "delta_pct", "per_video_delta"):
32         assert k in res
33     h = res["honest"]
34     assert {"variant_a_ci", "variant_b_ci", "paired_delta_f1", "segmentation"} <= set(h)
35
36     pd = h["paired_delta_f1"]
37     assert pd["mean_delta"] > 0.0 # Gate-A removes the FP ? F1 up
38     assert pd["ci_excludes_zero"] is True
39     assert pd["sign_test"]["wins"] == 2.0 and pd["sign_test"]["losses"] == 0.0
40
41     # C4: CONTROL over-segments (3 preds / 2 rallies = 1.5); Gate-A is 1:1.
42     assert h["segmentation"]["variant_a"]["segment_count_ratio"] == 1.5
43     assert h["segmentation"]["variant_b"]["segment_count_ratio"] == 1.0
44     seg_a = h["segmentation"]["variant_a"]
45     assert "f1@10" in seg_a and "f1@25" in seg_a and "f1@50" in seg_a
46
47     # CI block present per metric
48     assert "f1" in h["variant_b_ci"] and "ci_low" in h["variant_b_ci"]["f1"]
49
50
51     def test_honest_stats_false_restores_legacy_shape():
52         res = compare(_corpus(), CONTROL, GATE, honest_stats=False)
53         assert "honest" not in res
54         assert "delta" in res and "per_video_delta" in res # legacy fields still present
55
56
57     def test_record_experiment_carries_honest_delta(tmp_path):
58         res = compare(_corpus(), CONTROL, GATE)
59         row = record_experiment(res, path=str(tmp_path / "lb.json"),
60                                timestamp="2026-06-14T00:00:00Z")
61         assert "honest_delta_f1" in row
62         assert "sign_test" in row["honest_delta_f1"] and "ci_low" in row["honest_delta_f1"]
63

```

29. assistant (2026-06-18T07:55:46.448Z)

Now let me check whether the fusion loaders have their own tests, and verify the environment (can I run the tests to confirm green baseline?).

30. user (2026-06-18T07:55:47.931Z)

```

test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
=== fusion_compare test refs ===
tests/test_ablation.py
tests/test_audio_features.py
tests/test_experiment.py
tests/test_fusion_compare.py
tests/test_per_user_eval.py
tests/test_per_user_regression.py
tests/test_served_gate_a_regression.py

```

31. user (2026-06-18T07:55:50.402Z)

```

numpy 2.4.6
reportlab ok
pypdf ok
cv2 ok

```

32. assistant (2026-06-18T07:55:53.759Z)

numpy/reportlab/pypdf/cv2 are all available. Let me check that `test\_fusion\_compare.py` covers `load\_videos`, and run the four relevant test files to confirm a green baseline before proposing.

33. user (2026-06-18T07:55:54.372Z)

```
46: def test_load_videos_skips_windowing_only_in_mixed_corpus(tmp_path):
54:     videos = fusion_compare.load_videos(d)
58: def test_fusion_compare_reports_person_driven_lift(tmp_path):
62:     videos = fusion_compare.load_videos(d)
75: def test_fusion_compare_delta_stats_and_ci(tmp_path):
80:     res = fusion_compare.compare(fusion_compare.load_videos(d), iou=0.5)
90: def test_fusion_compare_needs_two_videos(tmp_path):
92:     videos = fusion_compare.load_videos(str(tmp_path))
98: def test_extract_video_assembles_shuttle_and_person(monkeypatch):
133: def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
165: def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
196: def test_derive_video_path():
```

34. assistant (2026-06-18T07:55:57.846Z)

`fusion\_compare.load\_videos` and `ablation.load\_videos` both have dedicated mixed-corpus skip tests. `fusion\_audio.load\_videos\_with\_audio` ? let me confirm and then run the baseline tests.

35. user (2026-06-18T07:55:59.127Z)

```
tests/test_audio_features.py:71:     videos = fusion_audio.load_videos_with_audio(d)
tests/test_audio_features.py:79: def
test_load_videos_with_audio_skips_windowing_only_in_mixed_corpus(tmp_path):
tests/test_audio_features.py:88:     videos = fusion_audio.load_videos_with_audio(d)
tests/test_audio_features.py:102:         fusion_audio.load_videos_with_audio(str(tmp_path))
tests/test_experiment.py:62:     (ablation.load_videos / fusion_compare.load_videos /
fusion_audio.load_videos_with_audio)."""
--- done ---
```

36. user (2026-06-18T07:56:25.636Z)

```
..... [ 64%]
...SS..... [100%]
110 passed, 2 skipped in 23.33s
```

37. assistant (2026-06-18T07:56:30.277Z)

All three loaders (`ablation.load\_videos`, `fusion\_compare.load\_videos`, `fusion\_audio.load\_videos\_with\_audio`) have dedicated tests including mixed-corpus skip tests. The 110 passing tests confirm a green baseline (2 skips are the golden-litmus tests needing `output/\*\_golden\_features.json`).

Now I have enough to assess. Let me verify one more detail: the duplicated PER-FILE feature-loading loop body across the 3 loaders, and check what `fusion\_golden` exports for the column-name constants to confirm a dedup is clean.

38. user (2026-06-18T07:56:32.642Z)

```
backend/eval/fusion_golden.py:34: SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
backend/eval/fusion_golden.py:36: PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
backend/eval/ablation.py:65: SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity",
"mean_velocity",
backend/eval/ablation.py:67: PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court",
"two_sided_motion", "mean_player_motion",
backend/eval/ablation.py:69: AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count",
"audio_hit_rate", "audio_hit_strength_mean")
```

```

backend/eval/ablation.py:423: cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) +
list(AUDIO_COLUMNS)
backend/eval/fusion_compare.py:24:from backend.eval.fusion_golden import PERSON_FEATURE_NAMES,
SHUTTLE_FEATURE_NAMES
backend/eval/fusion_compare.py:71: shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
backend/eval/fusion_compare.py:74:
feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),
backend/eval/fusion_audio.py:25:from backend.eval.fusion_golden import PERSON_FEATURE_NAMES,
SHUTTLE_FEATURE_NAMES, _derive_video_path
backend/eval/fusion_audio.py:95: feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES), threshold=threshold)
backend/eval/fusion_audio.py:98: feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES) + list(af.AUDIO_FEATURE_NAMES),
backend/pipeline/detectors/audio_features.py:15:AUDIO_FEATURE_NAMES = ["audio_hit_count",
"audio_hit_rate", "audio_hit_strength_mean"]

```

39. assistant (2026-06-18T07:56:39.045Z)

This confirms a subtle nuance for the dedup: the column-name constants differ between modules (``ablation.SHUTTLE_COLUMNS`` is a hardcoded tuple, ``fusion_golden.SHUTTLE_FEATURE_NAMES`` is derived from ``distill_local.FEATURE_NAMES``), and the audio-handling differs (ablation tolerates missing audio with ``.get(...,0.0)``; fusion_audio *requires* audio and raises). So a dedup must parameterize the column lists and the audio policy ? still behavior-preserving but more than a trivial extraction. This raises the dedup's risk slightly. I have what I need.

Let me do a final check on the ``_fps_fw`` / ``_fps_fw_from_prov`` helpers in golden_fixtures to verify the dead-code-ish candidates and assess the cleanest extraction seam for the PDF/reporting split in ablation.

40. user (2026-06-18T07:56:42.034Z)

```

backend/eval/harness.py:108:def format_report_text(report: EvalReport, show_failures: bool =
False) -> str:
backend/eval/served_gate_a.py:157:def format_table(results: List[ServedVideoResult], agg:
Dict[str, Any], iou: float) -> str:
backend/eval/served_gate_a.py:195: print(format_table(results, agg, args.iou))
scripts/nightly_regression.py:368:def format_report(baseline: Dict[str, Any], current: Dict[str,
Any],
scripts/nightly_regression.py:605: report = format_report(baseline, snapshot, result, tols,
date)
scripts/run_phase3_eval.py:39:from backend.eval.harness import evaluate, format_report_text,
report_to_dict
scripts/run_phase3_eval.py:159: print(format_report_text(report,
show_failures=args.show_failures))

```

41. assistant (2026-06-18T07:56:50.574Z)

Good ? the ``export_pdf``/``format_table``/``format_report`` functions in these three modules are only called within their own module + tests (no external importers outside the module). This confirms the reporting/PDF split seams are clean and self-contained.

I now have a complete, honest picture. Let me synthesize the proposals.

Summary of what I verified:

- `**golden_fixtures.py (1113L)**`: Very strong test coverage (test_golden_fixtures.py + test_golden_real_fixtures.py, ~50 tests covering replay, freeze, refusal gate, scan_forbidden, compare_expected, CLI). The file has natural module seams: synthetic generation, replay core, P1 de-attribution/freeze_real, manifest/loaders, compare, CLI.
- `**ablation.py (620L)**`: Strong coverage (test_ablation.py, ~30 tests incl. PDF via pypdf, CLI, load_videos). ``export_pdf`` (116L of reportlab) is a clean self-contained split.
- `**experiment.py (579L)**`: Comprehensive coverage. ``_is_fusion_schema`` already consolidated. No obvious dead code or oversized split ? it's cohesive.
- `**rally_seg_eval.py (380L)**`: Pure scoring core well-tested; `**reporting layer`

(`format\_report`, `\_format\_guard`, `\_ci`) and the `main()` CLI preset-wiring have NO tests.**
- **Cross-module dedup**: 3 near-identical `load\_videos` loops, but column constants + audio policy differ, so dedup needs parameterization (medium risk, but well-tested on all 3).

42. user (2026-06-18T07:57:39.133Z)

Output does not match required schema: /proposals/1: must NOT have additional properties,
/proposals/3: must NOT have additional properties

43. assistant (2026-06-18T07:57:42.220Z)

I added a `behavior\_preserving\_note` field that isn't in the schema. Let me fold those notes into the allowed fields and resubmit.

44. user (2026-06-18T07:58:17.641Z)

Structured output provided successfully

45. assistant (2026-06-18T07:58:34.434Z)

I've completed the read-only scoping of the eval god-modules cluster. Here is a summary of my findings.

What I verified

- Read all four cluster files in full plus their matching test files (`test\_golden\_fixtures.py`, `test\_golden\_real\_fixtures.py`, `test\_ablation.py`, `test\_experiment.py`, `test\_experiment\_honest.py`, `test\_rally\_seg\_eval.py`).
- Confirmed a green baseline: `110 passed, 2 skipped` across those six files (the 2 skips are golden-litmus tests needing `output/\*\_golden\_features.json`).
- Confirmed `numpy`, `reportlab`, `pypdf`, `cv2` are all importable here, so PDF/CLI tests actually run.

Four proposals (ranked)

1. **HIGH value / MEDIUM risk ? Split `golden\_fixtures.py` (1113L) ?
`golden\_real\_fixtures.py`.** The P1 de-attribution / `freeze\_real\_fixture` path (~290L) is the largest cohesive chunk and already has its own dedicated test file, so the split mirrors the existing test boundary. Strongly tested. The one caveat is a circular-import risk from the back-compat re-export (resolvable by placing the re-export at file bottom) ? that's why it's medium, not low.
 2. **MEDIUM / LOW ? Extract `export\_pdf` (116L reportlab) out of `ablation.py` ?
`ablation\_pdf.py`.** Pure presentation, lazily-imported reportlab, byte-for-byte move, covered by `test\_export\_pdf\_is\_valid\_nontrivial` + the CLI test.
 3. **MEDIUM / LOW ? `needs-tests-first` for `rally\_seg\_eval.py`.** Honest flag: the scoring core is well-tested but `format\_report`, `\_format\_guard`, and the entire `main()` CLI preset-override wiring (the `--vs` duplication + `\_apply\_inactivity` closure) have **zero** coverage. The real extract-method cleanup there should NOT happen until characterization tests land ? this proposal is tests-only (0 production change).
 4. **MEDIUM / MEDIUM ? Dedup the three near-identical `load\_videos` loaders** (`ablation`, `fusion\_compare`, `fusion\_audio`) into one parameterized helper. All three callers have dedicated tests including mixed-corpus-skip. Subtlety that bumps risk: the shuttle-column constants are different objects across modules (hardcoded tuple vs `distill\_local.FEATURE\_NAMES`-derived) and audio policy differs (tolerate-0 vs require-raise), so the helper must take column lists + audio policy as parameters.
- **Notably NOT proposed:**** `experiment.py` (579L) ? it's cohesive, comprehensively tested, and `\_is\_fusion\_schema` was already consolidated (single definition imported by all three loaders). No clean behavior-preserving split or dead code worth a PR there right now.

The relevant files (all absolute):

- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/eval/golden\_fixtures.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/eval/ablation.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/eval/experiment.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/eval/rally\_seg\_eval.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/eval/fusion\_compare.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/eval/fusion\_audio.py`